

Algorithm Benchmarking and Outcomes

**The Society Connectedness & Health Lab
Yale School of Public Health**

Church Closures Dashboard Project



Table of contents

Preparation of the 2026 Format Raw Data	2
Overview	2
Important Considerations	3
Ignored Files	3
Environment Reproducibility	3
Methods	5
Approach and Methodological Objectives	5
Limitations on Pre-Cleaning and Data Retention	6
Special Considerations for PO Boxes and Missing <code>address_line_1</code> Entries	6
Step 1: Nomenclature Standardization and Parquet Conversion	8
Step 2: Clean and Validate Addresses and Geolocation, and Annotate Records with	
Census Boundaries	11
LOOP PART A: Isolate Unique Candidate Addresses	12
LOOP PART B: Consolidate and Verify the Addresses	12
LOOP PART B.i.: Validate Addresses with USPS Database	13
LOOP PARTS B.ii-B.iii: String Similarity Matching for Unverified and	
Heterogeneous Address	16
LOOP PART C: Verify Geolocation with the US Census Bureau's Geocoder	
Database	21
LOOP PART D: Point-in-Polygon Spatial Assignment of Census Information .	23
LOOP PART E-I:	29
Compiling Results and Evaluating Import Quality	29
Step 3:	31
Results	32
Step 1: Nomenclature Standardization and Parquet Conversion	32
Step 2: Clean and Validate Addresses and Geolocation, and Annotate Records with	
Census Boundaries	32
Compilation Quality Control Checks	32
Overview of Address Completeness and ABI Filing Patterns	33
Address Validation and Consolidation Process	36
Address-Based Geocoding	39
Census Boundary Annotation Using Point-in-Polygon Spatial Joins	44
Step 3:	48

Appendix	49
All SIC Codes and Descriptions	49
Primary SIC Codes and Descriptions	50
Step 1 Final Result	50
State-Level Shapefile for Basemap	51
Missing Address Lines and PO Boxes By City	51
GEOID Prebuild Shapefiles	53
Core Area CBSA/CSA Prebuild Shapefiles	53
Core Area ZCTA Prebuild Shapefiles	54
Step 2 Final Result	55
Step 2 Final Result - Import Quality Checks	57
Step 2 Algorithm Quality Check Datasets - Import Quality Checks	58
Step 2 Algorithm Quality Check Datasets - Addresses	59
Step 2 Algorithm Quality Check Datasets - Census	62
Step 2 Algorithm Quality Check Datasets - Geocoordinates	65
References	68

Preparation of the 2026 Format Raw Data

Prepared By: Shelby Golden, M.S. from the Yale School of Public Health [Data Science and Data Equity](#) team

Date Created: August 7th, 2026

Date Updated: August 11th, 2026

Overview

The project objectives were as follows:

1. **REVIEW:** Evaluate the raw data and previously outlined cleaning, validation, and harmonization methods employed by [Dr. Insang Song](#) for accuracy and identify opportunities for improvement.
2. **PREPARATION:** Implement identified improvements to prepare the dataset for analysis and distribution to qualifying collaborators.
3. **METRICS:** Calculate the metrics necessary for dynamic visualization.

Additionally, the project Principal Investigator (PI), [Professor Yusuf Ransome](#), requested the pipeline be reproducible for future collaborators and readily adaptable to potential changes in project scope.

Following receipt of the raw data in Summer 2025, a review was conducted and findings were compared against the original methodology provided by Dr. Song. When an updated dataset was subsequently delivered in May 2026, the pipeline was expanded to accommodate two designated variations: the **2023 Format** and the **2026 Format**.

This document provides an overview of the methodological changes implemented following the initial prototype developed using the 2023 Format, and to accommodate the differences identified in the 2026 Format Review. While many of the fundamental methods were retained, notable changes to the step-wise ordering and consideration of edge cases were implemented in light of the prototype's algorithm performance and access to the High Performance Computer (HPC).

Results generated from this pipeline were used for the Fall 2026 [dashboard](#) developed by Gordon Tu.

Important Considerations

Selected code excerpts are referenced below. All snippets are drawn from scripts saved to the “[Code/](#)” directory, each prefixed by a descriptor indicating the step to which they apply:

Clean Raw Data_Step *: A discrete step in the methods pipeline outlined below. Additional classifiers may indicate whether the script was designed for use on the HPC and/or whether it applies to the 2023 or 2026 version of the raw data.

Ignored Files

Individual-level files are withheld from GitHub and Git tracking in accordance with the Data Use Agreement (DUA) with the data provider. Many of these are stored in `**/KEEP LOCAL/` directories or are explicitly listed under the “Project-Specific Exclusions” section of the `.gitignore` file.

Most files are available on the SOCAH Lab OneDrive for the project, while others are user-specific and do not need to be shared. If there are local-only files for this project, they will be added to the “Church-Closures-Dashboard GitHub LOCAL ONLY Files” directory. This directory will contain `README.md` files specifying the intended use and filepath for each file.

If you do not have access to the SOCAH Lab OneDrive, carefully follow the codebase instructions to establish the repository and replicate results on your own device. Raw data access instructions may vary depending on your institution.

Environment Reproducibility

Results were produced using R (v4.5.2), RStudio (v2026.01.1+403), and Quarto (v1.8.27). The `renv` package (v1.1.4) is included to reproduce the coding environment, capturing all relevant packages and their versions. If issues arise when running the scripts, verify that the environment is initialized and that the correct versions of R, RStudio, and Quarto are being used.

⚠ Compatibility

Updating packages, R, or Quarto may introduce compatibility issues with legacy code that will need to be resolved on a case-by-case basis.

First-time users should initialize the environment to install all required packages and ensure a reproducible coding environment. To do so:

1. In the command-line application (i.e., Terminal for Macs or Git Bash for Windows) navigate to the codebase file location.

Command-Line Interface

```
cd "FILE-PATH/Church-Closures-Dashboard/"
```

2. Launch the project by opening the *.Rproj file in RStudio.
3. In the R console, activate the environment by running the following lines of code.

Command-Line Interface

```
renv::restore() # Download packages and their version saved in the  
↪ lockfile.
```

i Note

If you are asked to update packages, say no. `renv` is intended to recreate the same environment under which the project was created, making it reproducible. You are ready to proceed when running `renv::restore()` gives the output:

RStudio Console

```
- The library is already synchronized with the lockfile.
```

If you experience any trouble with this step, you might want to confirm that you are using R (v4.5.2) in the RStudio (v2026.01.1+403) and Quarto (v1.8.27). You can also read more about `renv` in their [vignette](#)¹.

Methods

Approach and Methodological Objectives

The following pipeline was designed to address two primary goals:

1. Produce a cleaned and validated form of the raw data suitable for dynamic visualization at the finest regional resolution: block groups.
2. Associate each address with census boundaries defined during the 2000, 2010, and 2020 decennial years, including block group, tract, county, state, and ZIP Code Tabulation Areas (ZCTAs).

Documentation on the expected accuracy of geocoordinates provided in the raw data was not made available by the data distributor. It was therefore considered important to compare the raw geolocations against a trusted database in order to associate verified geolocations and assess the extent of error in the raw form. While not the most accurate approach, address-based database matching is a generally accepted method for confirming approximate geocoordinates².

Each address's geolocation coordinates can be used to annotate each entry with the respective decennial years census boundary via point-in-polygon geocoding. The U.S. Census Bureau TIGER/Line shapefiles containing the required census boundary information provide corresponding files for decennial years requested: 2000, 2010, and 2020. However, this technique can be computationally costly, resulting in maxing in memory limits or causing preclusively slow loading if not handled strategically.

During the raw data review, it was noted that typographical errors resulted in duplicate address entries, where the associated open/closure years remained unique. Ideally, these errors

would be cleanly reconciled to produce a concise dataset free of unnecessary redundancies. However, the presence of address variants is not precluding to the overall goals of this project. Mitigating these variations is nonetheless worthwhile to minimize unnecessary computational costs during geocoding and subsequent metric summarization across all addresses. Therefore, an attempt will be made to responsibly consolidate addresses using string similarity matching. Additional resolution is expected when address validation is done against a trusted database.

Limitations on Pre-Cleaning and Data Retention

It is important that simplifying the data cleaning and validation process does not necessitate the removal of potentially valuable or salvageable entries. Therefore, many of the algorithmic approaches employed were designed to accommodate complicated cases and missing data, with the aim of maximizing the number of usable final results. Quality control metrics were captured at the time of processing to support post-algorithm adjudication, back-tracking, and identification of values requiring removal prior to metric calculation.

Specific workarounds are described in each relevant methods section, with brief assessments of algorithmic performance reported in the respective [Results](#) section. The scope of **Step 2** was extensive enough that its findings have been compiled in a separate document dedicated solely to reporting these results. Only key insights and conclusions are presented here.

Special Considerations for PO Boxes and Missing `address_line_1` Entries

The raw data review of the 2023 and 2026 Formats revealed several notable special cases requiring specific handling. These are described in the respective sections of [Step 2: Clean and Validate Addresses and Geolocation](#), and [Annotate Records with Census Boundaries](#). Notably, approximately 14% of businesses had at some point filed under a PO Box, and approximately 3% had at some point submitted a record with no `address_line_1` value (`address_line_1 == NA`; no blank strings were observed). For context, each entry in the dataset represents a unique combination of address and year over 1,080,764 unique ABI, with no year repeated within a given business.

Later in the analysis, religious organizations are classified as closed if they did not file for four consecutive years during the observed period; organizations that subsequently resumed filing are classified as reopened. Closures are further differentiated into two categories: those that stopped operations entirely and closures due to relocation. Relocated organizations remained operational but moved outside the community, thereby resulting in a loss of social benefits to

that community.

Addresses filed under a PO Box or missing an `address_line_1` value present challenges for generating these metrics. Dr. Insong's method addressed this by removing all PO Box entries while retaining the remaining addresses for each business. However, since reported dates were unique across entries, this was expected to skew results. The Summer 2025 prototype methods sought to associate each PO Box with a nearby physical address (both spatially and temporally) to serve as a proxy for the organization's location on those dates. This relied on the assumption that PO Boxes are opened near the business's operating location, which was later deemed too strong an assumption to adopt.

For these reasons, under the following three conditions, all entries associated with a business are removed:

- Geocoordinates are absent and could not be recovered through address-based validation, preventing spatial joining (0.39% missing before **Step 2** and 0.03% missing after validation/cleaning).
- `address_line_1` is missing, preventing verification that the geocoordinates correspond to the institution itself rather than a general location within that ZIP code (3% ABI).
- PO Box used: the physical location of the business, and consequently its presence or absence within the community of impact, is ambiguous (14% ABI).

Table 1 shows the distribution of ABIs impacted by any one of these conditions. Not shown are results indicating that longitude and latitude were always missing together, affecting 0.4% of ABIs. These results demonstrate very little overlap between conditions, suggesting that approximately 17% of ABIs, nearly one-fifth of ABI, could potentially qualify for removal.

Table 1: Percentages of ABI with Address and Geolocation Considerations

NA Address = FALSE			NA Address = TRUE		
NA Lon	PO Box		NA Lon	PO Box	
	FALSE	TRUE		FALSE	TRUE
FALSE	83.51	13.48	FALSE	2.38	0.24
TRUE	0.36	0.02	TRUE	0.02	0.00

Simply removing ABIs with these characteristics introduces a potential source of bias, especially among those filing under a PO Box. Table 2 summarizes the share of ABIs affected by PO Box filings and missing `address_line_1` across U.S. cities, with some reaching 100% of ABIs impacted. Figure 1 maps these values spatially, confirming that some communities are

disproportionately affected by this data loss. Note that 10.6% and 12.6% of cities could not be matched to TIGER/Line Shapefile features imported via the `tigris` package.

Table 2: Summary Statistics of ABIs with Select Characteristics by City

	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
NA address line 1 (%)	0.09	2.56	6.25	14.93	17.65	100.00
PO Box (%)	0.26	15.38	33.33	42.16	63.64	100.00

The agreed-upon mitigation strategy is to remove an ABI only when the selected date range includes an address with a missing `address_line_1`, a PO Box filing, or an address without geocoordinates and, consequently, no census boundary mapping. The number of ABIs removed under each condition will be reported on the dashboard to inform users of the extent of data loss attributable to each condition.

Step 1: Nomenclature Standardization and Parquet Conversion

💡 Computational Dependencies and Custom Functions

All packages required for this step are documented in the **SET UP THE ENVIRONMENT** section of [“Clean Raw Data_Step 1_2026 Format.R”](#). Custom functions used throughout the script are in [“Support Functions/For Step 1_2026 Format.R”](#).

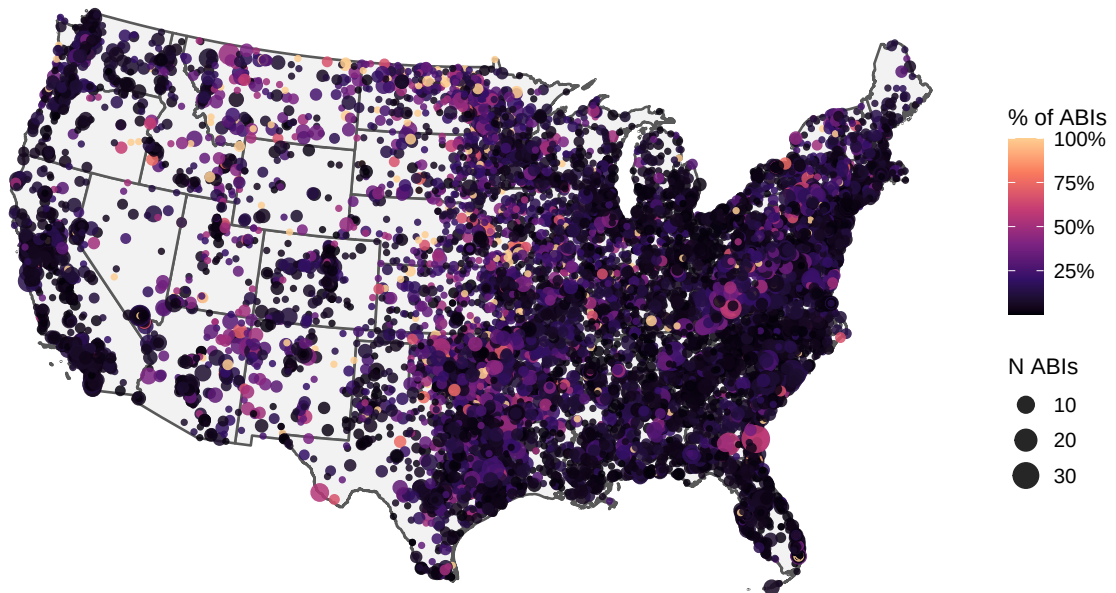
The 2026 Formatted data included a number of additional metadata fields; some were directly relevant to the analysis done here, while many others were not directly relevant or were undefinable. Further details can be found in the [“Review_2026 Format.pdf”](#) report. It was decided to retain all metadata fields in accordance with good data science practices, while limiting further validation and processing steps to only those fields relevant to the immediate analysis.

In **Step 2**, address, geolocation, and census boundary variables are thoroughly processed through sequential validation and cleaning measures. Prior to this step, a few additional metadata fields were reformatted but not validated, as the data provider is treated as the known authoritative source of these categorical attributes.

The North American Industry Classification System (NAICS) code is typically six digits long; however, the data provider appended an additional proprietary two-digit code whose encoding

ABIs with Any Missing Address Line 1 by City

Color = % of ABIs; size = N ABIs



ABIs using PO Boxes by City

Color = % of ABIs; size = N ABIs



Figure 1

is unknown. These were separated into two columns to clarify the association between the NAICS code and its description, distinct from the additional two-digit code. This yielded three new columns: `primary_naics6_code`, `naics6_descriptions`, and `primary_naics2_code`, the last of which has no associated description.

The SIC codes appear in a primary column, limited to 42 unique values, and four additional overflow columns representing over 2,500 unique attributes, such as denomination, functional capacities (e.g., screen printing, concert venues, stables), and services (e.g., clothing, food banks, substance abuse support). Three assumptions associated with these columns were validated: (1) the additional SIC code columns are overflow columns, where preceding columns are non-NA, indicating progressive filling; (2) codes and descriptions are mapped one-to-one; and (3) codes are always associated with a non-NA description.

Assumptions were tested using the custom function `sic_overflow_audit()`, which generates all relevant diagnostics. Minor inconsistencies between descriptions associated with the same SIC code were manually revised across all SIC columns using the custom function `clean_sic_descs()`. Any SIC code sometimes associated with a description and sometimes NA was revised to use the available description using the custom function `set_sic_desc_for_code()`. If multiple descriptions were associated with the same code, one description was chosen at random using the custom function `clean_sic_codes()`.

As a final step, the dataset was converted from the CSV output of the Data Axle database to Parquet format using the `arrow` package³. This columnar format stores large datasets more efficiently, enabling faster read and write operations. It also supports database-style querying, allowing users to limit the amount of data loaded into memory at any given time and making better use of available RAM.

Additional details regarding the algorithm's performance outcomes can be found in [Results - Step 1: Nomenclature Standardization and Parquet Conversion](#).

Step 2: Clean and Validate Addresses and Geolocation, and Annotate Records with Census Boundaries

💡 Computational Dependencies and Custom Functions

All packages required for this step are documented in the **SET UP THE ENVIRONMENT** section of [“Clean Raw Data_Step 2_2026 Format.R”](#). Custom functions used throughout the script are in [“Support Functions/For Step 2_2026 Format.R”](#).

The algorithm implementing the described methods has been separated into a dedicated HPC script, contained in [“Clean Raw Data_Step 2 HPC v2_2026 Format.R”](#), which includes its own **SET UP THE ENVIRONMENT** section. **PART A: UTILIZING THE HPC** further describes how to reproducibly run the script within the HPC environment as both a live session or batch array. The preconfigured shell script used for deploying batch arrays is [“Validation SLURM_2026 Format.sh”](#).

Note that the comparative HPC script [“Clean Raw Data_Step 2 HPC v1_2026 Format.R”](#) reflects the initial approach assuming unrestricted access to the USPS API. However, in Spring 2026, the USPS announced an update from version 3.2.3 to 3.3.1 with revised terms of use and a new pricing tier that took effect August 1st, 2026. In response, the algorithm was modified to more flexibly support agnostic address compression based on string similarity, and to control API usage via a boolean flag indicating whether the API should be used and, if TRUE, a query limit specifying how many addresses were to be validated in that run.

The methods are comparable between the two versions, with the logical flow of the address validation and compression algorithm being the principal difference. As a consequence of the API terms of use update, v2 was used to process the entire dataset. Users should therefore treat v1 as archived for posterity and v2 as the version to be used going forward. The specifics of v1 are not discussed here.

With access to the HPC, all sequential steps that validate and compress addresses, use them for address-based geocoding, and annotate unique ABI and address entries with census boundaries are completed within a single algorithm. Each macro component is prefaced with **LOOP**, corresponding to the respective section of the HPC script.

Each validation section requires user-specified configurations or pre-analysis setup. The specifics of these are described in each relevant section, but must be completed prior to running the algorithm to ensure it operates correctly and within the constraints of the project’s available

resources.

LOOP PART A: Isolate Unique Candidate Addresses

Addresses associated with each individual ABI are processed simultaneously. First, each ABI's unique address is identified and saved into `candidate_addresses`, which retains the key address fields. For each unique address, the earliest and latest years it appears are recorded, the number of entries with valid longitude/latitude is counted, and the average longitude and latitude (among valid entries) is computed. Additionally, the boolean `do_api`, indicating whether the USPS API should be used, is carried forward when present. These candidate addresses are then passed through all subsequent processing sections.

If `address_line_1` is missing, the entry cannot be processed for address validation or address-based geocoding. As noted earlier, any given geocoordinates for these records cannot be assumed to represent the business's physical location. Although the algorithm can still annotate these entries with census boundaries, this does not mean they are suitable for mapping closure results. To prevent errors when `address_line_1 == NA`, these records are separated from those with an `address_line_1` so they can be validated and then condensed.

! Note on Geocoordinate Accuracy for Entries Missing the Address

While entries lacking an `address_line_1` but have a valid geocoordinates are annotated with census boundaries, this does not imply that the geocoordinates accurately reflect the physical location of the business. This measure is intended solely to allow the algorithm to run without error, and affords users the opportunity to adjudicate these entries at a later stage if desired.

LOOP PART B: Consolidate and Verify the Addresses

Address heterogeneity arising from typographical errors was observed in the 2023 Format and is assumed to exist in the 2026 Format. In the 2023 version of this stepwise process, addresses were first compressed based on similarity before validation. This ordering was necessary because the workflow was run locally, and reducing the number of unique addresses to validate helped expedite the USPS API step.

However, a post-assessment of the algorithm’s outputs showed that the original compression step induced duplications in years-open and years-closed measures. This largely occurred because the same `address_line_1` was reported with differing metadata (e.g., city or ZIP code). After validation, a secondary address-compression step was run on qualifying entries where this issue was detected, using the corrected, standardized address information to recompress records.

With access to the HPC, the compute constraints of address verification at scale became tractable. The revised workflow reflected here resolves heterogeneity in two stages: (1) USPS API verification, which standardizes addresses and reconciles unverified records to their closest verified match using string similarity; and (2) agnostic clustering (a fallback when verification is unavailable or fails), which groups similar addresses by string similarity and resolves conflicts within each cluster to reduce redundant entries.

The same algorithm is used at both points where address string similarity is evaluated, but the way “best” matches are triaged differs between the two stages. To prevent the table from expanding when multiple best matches are identified, a single match is forcibly selected by ranking candidates according to a predefined set of criteria and choosing the top-ranked result.

LOOP PART B.i.: Validate Addresses with USPS Database

Excessive deviation from standard address formatting has the potential to contribute to failed matches with the [U.S. Census Bureau’s Geocoder API](#), used in **LOOP PART C** to validate the geocoordinates associated with each address ⁴. To mitigate this risk, addresses were first validated against the [USPS Address API](#) to retrieve the preferred, standardized form of each address prior to geocoding ⁵.

i Alternative Validation Resources

These APIs were chosen primarily for their relative simplicity of setup and use, their suitability for automation and parallelization within the HPC environment, and their lack of associated cost. However, as described in their respective results sections, these processes were unable to validate as many records as anticipated. Furthermore, as they are separate systems, they required sequential processing, whereas alternative resources can offer both address and geolocation validation within a single query. The following alternative databases were considered for validating the listed physical

addresses and associated geolocations:

- Environmental Systems Research Institute, Inc. (Esri) StreetMap Premium, including the locally hosted version available to Yale affiliates
- Google Maps API

These sources were ultimately not used due to a combination of limiting factors. Under the applicable Data Use Agreements (DUAs), it was necessary to confirm that no API usage would violate the terms of those agreements. The USPS and U.S. Census Bureau APIs were approved following coordination with the relevant security review processes; however, accessing the Esri online database or Google Maps API required additional review.

The review process was initiated for the Google Maps API given its anticipated capacity to efficiently validate addresses and geolocation. However, the review extended beyond the project timeline, rendering those resources inaccessible. Additionally, Esri requires manual adjudication of alternative address suggestions, which would have been prohibitively time-intensive. Researchers from the [Yale Center for Geospatial Solutions \(YCGS\)](#) further noted that the online Esri resource is more current and robust than its locally hosted counterpart, suggesting that the local version would offer little advantage over the USPS Address database in terms of address validation quality.

USPS API Version 3.3.1 Update: Compatibility and Pricing

As of August 1st, 2026, the USPS upgraded the API from version 3.2.3 to 3.3.1 and updated their terms of use to include tiered pricing for API access.

Users should be aware that the algorithm reported here was designed for version 3.2.3 and may not be fully compatible with the updated API without modification. Additionally, the 2023 Format implementation assumes unrestricted access to the API, whereas the 2026 Format implementation was updated with a version 2 allowing users to specify the number of addresses queried against the USPS API, enabling cost management.

In response to the updated terms of use, the workflow was revised to give users more control over address verification. All settings are outlined and explained in **SUBSECTION B3: Optional USPS Address Verification (API Setup and Quota Controls)**. Users can set the boolean `verify_addresses` to indicate whether verification should be run, and API credentials are imported only when `verify_addresses == TRUE`.

Users can also define an address-count quota via `set_quota`. This quota is then applied through an if/else selection and custom function `usps_quota_mark_api()` to determine which ABIs are sent for verification. Selection is made at the ABI level: if an ABI is chosen, all of its addresses are verified, and no partial ABI selections are allowed. As a result, `set_quota` functions as an upper bound on the number of addresses verified (the total may fall below the quota to avoid splitting an ABI). The if/else conditionals generate a message indicating whether the API will be used, how many ABI entries will be validated, and whether addresses were randomly sampled. A set seed allows users to reproduce any randomized selections.

When `verify_addresses == TRUE` the algorithm queries the USPS Address API using the API specifications, methods, and sample processes outlined in the official [developer documentation](#) and the [USPS API Examples GitHub](#) repository^{5,6}. Pre-configured customer key and secret credentials are used to generate an API token for each address query via the custom function `generate_usps_token()`. The raw address fields, address lines 1 and 2, city, state, and five-digit ZIP code, are submitted to the database, and the response JSON is parsed to extract validated address components into discrete columns via the custom function `validate_usps_address()`. Both functions utilize the `httr` and `jsonlite` packages for API interaction^{7,8}.

If the initial USPS validation did not return a match, one fallback strategy was applied using the recommended city associated with the five-digit ZIP code. If this also failed to produce a match, the query returned “No address match found”.

This fallback was particularly important for the 2023 Formatted dataset. Initial assessment showed that ZIP code fields had been converted to numeric values at some point, stripping leading and trailing zeros. Before querying, ZIP codes were normalized back to the expected five-digit format, but any padded zeros were added arbitrarily. Although the 2026 Formatted dataset correctly preserves ZIP codes as character variables, this step was retained as a precaution in cases where the provided city does not align with the ZIP code and ZIP codes got mistakenly converted to numeric.

In the **LOAD IN THE DATA** section, users can indicate whether ZIP codes were retained as character strings (i.e., no flanking zeros were stripped) by setting the boolean `zip_codes_character`. When `zip_codes_character == TRUE`, the fallback uses the custom function `get_city_info()` to retrieve the recommended city for a given ZIP code. The resulting valid ZIP code–city pair is then substituted for the raw values and the USPS query is resubmitted. The ZIP–city lookup table is built from the SimpleMaps Basic [United States Cities Database](#) (version 1.90) using the custom function `build_zip_city_lookup()`⁹.

If ZIP codes are indicated as having lost flanking zeros, candidate ZIP code variants are first generated using the custom function `make_zip5_candidates()`. For example,

`make_zip5_candidates("01230")` would generate “01230”, “00123”, and “12300”. The custom function `get_city_info()` then retrieves the recommended city associated with each candidate ZIP code. Valid ZIP code–city pairings are subsequently substituted for the raw values to resubmit the query, though the algorithm attempts only the first valid pairing returned by `get_city_info()`.

Five quality-control columns were used, including: (1) a boolean indicating whether the address was validated; (2) a character field indicating which attempt succeeded (e.g., “First try” or “With city correction by zip5”); and (3) the USPS API query status. The status codes were decoded using the custom function `usps_status_group()`, based on the expected outcomes defined in the USPS API documentation.

i API Credentials and Configuration

Users who wish to utilize this resource for their own work will need to register for a USPS Developer account, sign the license agreement, and configure their own individual API client key and secret.

The client key and secret must be kept private and must not be shared. Instructions for configuring these credentials are provided in the header of [“Code/Clean Raw Data_Step 2_2026 Format.R”](#).

LOOP PARTS B.ii--B.iii: String Similarity Matching for Unverified and Heterogeneous Address

The same underlying algorithm is used whenever string-similarity matching is performed. However, best-match selection differs by phase (matching with unverified addresses vs. “agnostic” matching). In both phases, candidate pairs are evaluated under two match classes: “exact” and “fuzzy.”

Exact matches. Exact matches compare `address_line_1` using threshold $\delta = 0$ and are treated as valid regardless of associated address metadata. In these cases, its assumed the likelihood that the records represent separate addresses due to a true physical move is low. Accordingly, the geolocation test is overridden and the match is retained.

Fuzzy matches. Fuzzy matches compare `address_line_1` using threshold $\delta > 0$ but ≤ 0.2 . For these match pairs, the longitude and latitude differences are required to fall within 0.02 degrees each. If the geolocation test fails, the match is rejected and the records are kept

separate.

Verified-Address Matching

This step occurs after addresses are queried against the USPS API. Unverified addresses are triaged by attempting to link each record to a verified address using `address_line_1`.

The algorithm first attempts an exact match by selecting the verified candidate whose years-open interval overlaps (or touches) the record's window, or—if none overlap—the candidate with the smallest year-gap; geolocation checks are skipped. If multiple candidates remain, ties are broken by preferentially selecting the closest match occurring before the window, and then after, then forcing only a single top candidate is retained.

If no exact match exists, the algorithm attempts a fuzzy match using the same years-open selection rule but requiring geolocation validation; records that fail validation remain separate. Fuzzy matching is run iteratively starting at $\delta = 0.2$ and then applying progressively more stringent similarity thresholds until each unmatched address resolves to a single best candidate; if no match is found at any threshold, the record is passed to the next section.

Agnostic Matching

This step runs after addresses are queried against the USPS API and unmatched records remain, or when no address validation is performed (i.e. either `verify_addresses == FALSE`, or `verify_addresses == TRUE` but `all(unver_candidates$do_api == FALSE)`). Because there is no predefined pool of verified candidates, the algorithm selects best matches agnostically from the addresses available within each cluster.

Within each exact `address_line_1` group, matching is restricted to records sharing the same expected city for the ZIP code, as defined by the lookup table produced by `build_zip_city_lookup()`. Using the custom function `pick_best_by_city()`, rows are split into trusted anchors where the reported and expected city are the same (`city_match == TRUE`) and conflicting records (`city_match == FALSE/NA`). If no trusted anchor exists, the best conflict row is promoted to serve as an anchor. Each record is then linked to a single best anchor, prioritized by (1) years-open overlap (or, if none overlaps, the smallest year-gap), (2) presence of geolocation, (3) smallest lon/lat difference across candidates, and (4) a stable tie-break. The geolocation test is bypassed and recorded as an override. Records with no anchor in their city are left for the fuzzy pass.

For fuzzy matching, similar strings are clustered and one anchor per validated city is chosen.

Anchors are preferred in order: presence of an exact match from the prior step, earliest year recorded open, then a stable tie-break; if no prior match exists, the “best” anchor is chosen based on geolocation pass rate, mean year gap, and geolocation coverage. Candidate pairs are rejected when both sides have coordinates and either longitude or latitude differences exceed 0.02 degrees. If either record in the pair is missing coordinates, the missing values are ignored and the test is recorded as "PASS; no Lon", "PASS; no Lat", or "PASS; no Lon/Lat" to indicate which geocoordinate is absent. Fuzzy matches are applied only when `geo_valid == TRUE`; otherwise records remain unmatched, with provenance recorded as `address_matched = "Fuzzy match"` and `match_geolocation_test` indicating whether the geolocation check was evaluated/passed.

For fuzzy matching, similar strings are clustered with $\delta = 0.2$ and one anchor per validated city is chosen, first preferring any exact matches created in the prior section. If no exact match is available, the anchor is selected based on how well each candidate represents the rest of the pool using the custom function `pick_best()`. This algorithm chooses the record whose coordinates agree with the largest number of other records (within the longitude/latitude difference < 0.02 degrees) and, if needed, the record whose years-open range overlaps with the most others. If ambiguity remains, it selects the record with the smallest overall year separation from the remaining records, then favors entries with a ZIP+4 and a larger number of records with valid geocoordinates for that address, and finally applies a stable alphabetical tie-break on the combined address string.

Fuzzy Matching Algorithm

Different uniquely listed address line 1 (`address_line_1`) strings for a given business (`abi`) were fuzzy matched using the Jaro-Winkler method available through `stringdist(method = "jw")` in the custom function `find_similar_addresses()`¹⁰. This edit distance method measures string similarity by accounting for the proportion of characters shared between two strings and the number of transpositions needed to place matching characters in the same order, normalized by string length^{11–13}. Scores range from zero to one, where a score of zero indicates identical strings and smaller values reflect greater similarity.

Prior to processing, addresses were normalized in the custom function `preprocess_address()` by converting all characters to lowercase, normalizing spacing and punctuation, removing non-alphanumeric characters (with the exception of commas and spaces), and trimming extraneous whitespace.

$_n C_2$ pairwise comparisons between n unique normalized `address_line_1` strings produce a symmetric $n \times n$ string distance matrix, that is $d(i, j) = d(j, i)$ and the diagonal entries

$d(i, i) = 0$ represent the trivial self-similarity solution. To reduce redundancy, the algorithm evaluates only one orientation of each pair and omits self-similarity comparisons, computing only the upper triangle of the matrix. Addresses with a pairwise Jaro-Winkler score below a set threshold, δ , are saved as a qualifying match. The matching threshold $\delta > 0$ and ≤ 0.2 was determined empirically through testing on a sample of addresses, confirming sufficiently accurate grouping of similar entries without the erroneous inclusion of dissimilar ones.

i Computational Performance

The time complexity of this operation is $\mathcal{O}(n^2 \times l)$, where n is the number of unique addresses and l denotes the computational cost of the Jaro-Winkler algorithm for a string of length l ^{13–15}.

Because the algorithm operates solely on `address_line_1` entries, string lengths remain relatively short, making the contribution of l negligible. Additionally, because the number of unique addresses is expected to be bounded by the number of dates in the observed period, the computational demand of this method is not expected to be a limiting factor in the algorithm.

Consider the following example of a business that filed under two distinct addresses, each of which was reported with slight variations, resulting in five unique address strings.

Node	Address
1	123 Main Street
2	123 Main St
3	123 Main St.
4	456 Oak Avenue
5	456 Oak Ave

Using `stringdist(method = "jw")` the pairwise distances are computed for all ${}_5C_2 = 10$ unique address pairs. This yields the following upper-triangular distance matrix:

	1	2	3	4	5
1	—	0.08	0.09	0.41	0.38
2		—	0.05	0.39	0.35
3			—	0.40	0.36
4				—	0.11
5					—

Pairs falling below the threshold $d(i, j) \leq \delta$ are considered candidate matches and recorded as edges in an undirected graph, where each node represents a unique address. This graph is stored as an adjacency list, like the one shown below¹⁶:

Node	Address	Nearest Neighbors	JW Distances
1	123 Main Street	{2, 3}	{0.08, 0.09}
2	123 Main St	{1, 3}	{0.08, 0.05}
3	123 Main St.	{1, 2}	{0.09, 0.05}
4	456 Oak Avenue	{5}	{0.11}
5	456 Oak Ave	{4}	{0.11}

A Depth-First Search (DFS) algorithm in the custom function `find_components()` is applied to the adjacency list to identify connected components, where each component forms a cluster of similar addresses.

Algorithm 1 Depth-First Search — Connected Component Identification

Require: Adjacency list `address_graph`, node set $\{1, \dots, n\}$

- 1: Initialize an empty list of clusters
 - 2: **for** each node i in $\{1, \dots, n\}$ **do**
 - 3: Begin a new traversal from node i
 - 4: Initialize an empty cluster
 - 5: **repeat**
 - 6: Add the current node to the current cluster
 - 7: **if** the current node has an unvisited neighbor **then**
 - 8: Move to an unvisited neighbor and continue
 - 9: **else**
 - 10: Backtrack to the previous node
 - 11: **end if**
 - 12: **until** no unvisited neighbors can be reached in the current traversal
 - 13: Record the cluster
 - 14: **end for**
 - 15: Remove duplicate clusters
 - 16: **return** list of unique clusters
-

One candidate address is selected from each cluster using the triage rules described above to serve as the best match and anchor for associating the other addresses in the pool. Because the reliability of string-based matching can vary with string length, an additional geographic validation step is applied: for each matched set, the absolute range in reported longitude and

latitude is computed as the difference between the maximum and minimum values.

For reference, one degree of longitude is approximately 69 miles (111 kilometers), while one degree of latitude varies with proximity to the equator, averaging approximately 54 miles (87 kilometers) across the contiguous United States^{17,18}. The length of a typical U.S. city block similarly varies, with common estimates ranging from 100 to 200 meters^{19,20}. Based on these benchmarks, address sets in which both the longitudinal and latitudinal differences exceeded 0.02 degrees (≈ 2 kilometers or ≈ 1.4 miles) were flagged as invalid matches and retained as distinct addresses in the final dataset.

This approach assumes that reported geolocations for each address are accurate to within the specified range with sufficient consistency to be reliable. As the primary objective of this method is to reduce address reduplication and improve downstream algorithmic performance, variations that may undermine this validation step are not considered to meaningfully impact the final results. When uncertain, the method is designed to err on the side of caution by retaining addresses as distinct rather than risk consolidating genuinely separate addresses that share strong surface-level similarities.

LOOP PART C: Verify Geolocation with the US Census Bureau's Geocoder Database

The dashboard visualizes aggregated addresses at multiple census geographies—block group, tract, county, state, and ZIP Code Tabulation Areas (ZCTAs). While small location differences have little impact on county-level summaries, they can substantially affect assignment to finer boundaries such as block groups and tracts.

In the 2023 Format, results from “Step 1: Address Deduplication via String Similarity Matching” showed that geocoordinates for records sharing the same `address_line_1` (and thus potentially the same physical location) sometimes differed by as much as 3 degrees. At mid-latitudes, a 3-degree difference is roughly 150–175 miles—about the distance between New Haven, CT and Philadelphia, PA.

To mitigate these errors, addresses are geocoded using the [U.S. Census Bureau's Geocoder API](#) to retrieve the coordinates assigned to each address, prioritizing verified addresses when available⁴. The API's documentation does not specify which part of the building the coordinates represent, such as the delivery entrance or rooftop centroid²¹. All available address fields (address line 1, city, state, and five-digit ZIP code) were used to find matches, prioritizing verified addresses over raw, given values.

Raw address fields are submitted to the Census Geocoder API, and the response

JSON is parsed to extract validated address components into discrete columns via the custom function `validate_geocoordinates_geoCoder()`. This function internally calls `call_census_geocoder()`, which relies on the `httr` and `jsonlite` packages for API interaction^{7,8}. The geocoding process first builds an ordered list of benchmark/vintage candidates, then iterates through them in priority order until a successful match is found.

In **SUBSECTION B2: Set Geocoder Search Priorities**, users specify the ordered list of benchmark/vintage pairs the geocoder should try. If users prefer not to use the default triage list, **SUBSECTION A2: Set Geocoder Search Priorities** in “Clean Raw Data_Step 2_2026 Format.R” provides the custom function `census_geo_show_options()`, which retrieves all available benchmark and vintage options that can be used for configuration.

RStudio Console

```
# Construct a prioritized table of benchmark/vintage search criteria,
# where earlier rows take precedence.
spec <- tibble::tibble(
  benchmark_name = c("Public_AR_Census2020", "Public_AR_Census2020",
                    "Public_AR_Current", "Public_AR_ACS2025"),
  vintage_name   = c("Census2020_Census2020", "Census2010_Census2020",
                    "Current_Current", "Current_ACS2025")
)

# Construct the "tries" input for `validate_geolocation()`
geocoder_census_tries <- census_geo_make_tries(spec)
```

For each candidate, the vintage is resolved to a numeric ID using `resolve_vintage_id()` — leveraging a per-run cache and skipping any candidates that fail due to lookup, network, HTTP, or parsing errors — before constructing a request URL, calling the API, and logging the outcome. An example query URL constructed from parsed address components using the custom function `build_addr_geo_url()` is shown below.

RStudio Console

```
httr::modify_url(
  "https://geocoding.geo.census.gov/geocoder/geographies/address",
  query = list(
```

```
format      = "json",  
benchmark  = benchmark,  
vintage     = vintage,  
street      = street,  
city        = city,  
state       = state,  
zip         = zip  
)  
)
```

The API sometimes returns multiple possible matches. If there are multiple candidates, the custom function `select_best_match()` first tries to resolve the choice using the ZIP code by preferring a candidate whose ZIP uniquely matches the input ZIP (treating ZIP+4 as the 5-digit ZIP). If ZIP does not identify a single candidate, it compares the input address text to the candidates' matched-address text using `find_similar_addresses()`, progressively tightening $\delta = 0.2$ until one candidate is most clearly paired with the input. If that still does not produce a clear winner, it defaults to the first candidate in the remaining list.

LOOP PART D: Point-in-Polygon Spatial Assignment of Census Information

Refer to [Appendix - GEOID Prebuild Shapefiles](#), [Core Area CBSA/CSA Prebuild Shapefiles](#), [Core Area ZCTA Prebuild Shapefiles](#) for information on where to find the code that generates these results, as well as the results codebook.

The data spans three decennial years: 2000, 2010, and 2020. Because users can select varying date ranges and may expect temporally accurate census boundary mappings, each address must be annotated with the census boundaries associated with all three. Once annotated, entries can be aggregated to the census boundary identifier from a given decennial US Census Bureau's [TIGER/Line Shapefile](#) release²². This enables results to be spatially projected onto maps via the decennially accurate demarcations of census boundaries.

Accurate spatial depiction of these trends across census boundary levels requires consideration of varying degrees of locational error. Counties and states represent larger areas, making them less sensitive to imprecise locational information and less prone to boundary changes between decennial periods. In contrast, smaller boundaries such as block groups and tracts can change substantially between decennial periods and are more sensitive to locational errors. Ensuring accurate fine-level annotations for historical decennial years is therefore an important consideration.

The preceding sections focus on validating and improving the geocoordinates associated with each business through address-based geocoding. When attaching census boundary identifiers to these records, there are two fundamental approaches:

1. Referencing a database with decennial boundary information already associated with each address.
2. Assigning decennial boundaries to each address via point-in-polygon spatial joining of verified longitude and latitude coordinates against reliable spatial representations of map boundaries, known as polygons.

The second approach is the standard of practice, ensuring all desired decennial boundary information is correctly assigned to the provided geocoordinates. Its accuracy is limited only by the reliability of the available address coordinates and boundary shapefiles. Figure 2 demonstrates how geocoordinate points are interpreted by a spatial join to polygon geometries, such as county-level census boundaries, geographically.

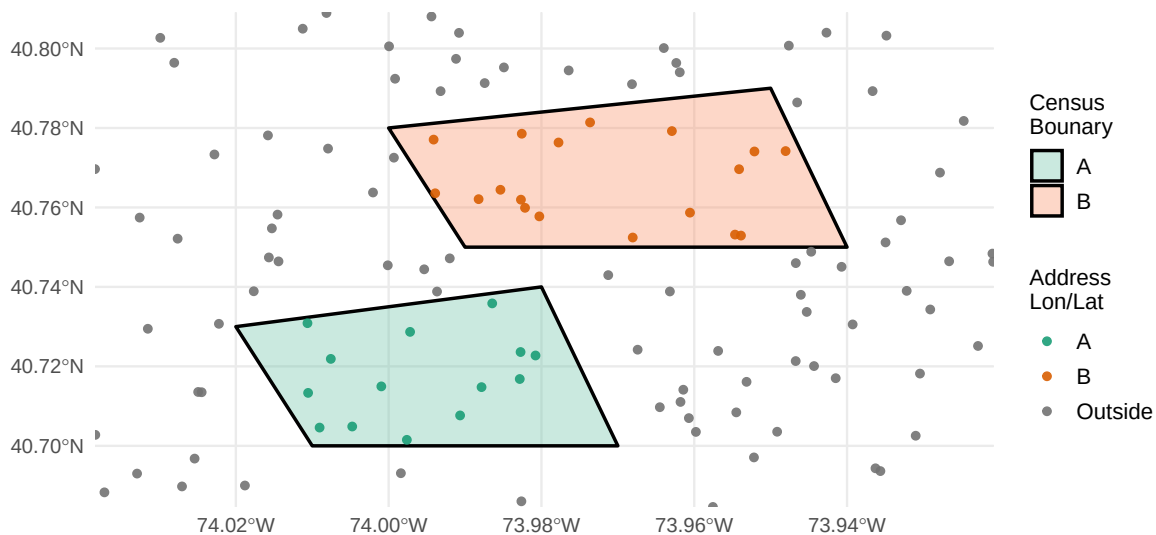


Figure 2: Illustration of Point-in-Polygon Spatial Joining

i Computational Performance

The time complexity of a naïve spatial join for a single decennial period is $\mathcal{O}(N \times M)$, where N denotes the number of points and M the number of polygons^{14,23}.

This can create significant time lags when associating multiple addresses across many boundaries, such as block groups or tracts in a densely populated region. Steps leading up to the point-in-polygon operation reduce the number of comparisons by consolidating addresses and filtering regions down to the state level. The county and other census boundaries were not used for filtering, as they were assumed to vary between decennial periods.

Other strategies exist that can improve the speed of this operation. For example, spatial filtering or indexing can reduce the complexity to $\mathcal{O}(N \log(N) \times M \log(M))$ ²³. Due to time constraints developing the process, these were not explored, and some entries, particularly those spanning denser census regions with more addresses, took orders of magnitude longer to compute than others.

While developing the pipeline for the 2023 Format, three sources of census boundary information were explored, all of which involved API calls to retrieve the requisite information at the time of annotation. This process exacerbated what was already a computationally costly spatial join operation, and was expected to worsen once the algorithm was expanded to include block group-level census codes, which are notoriously slow to query. Additionally, the `tigris` process exhibited issues retrieving data for the 2000 and 2020 decennial years.

To improve performance and accuracy in preparation for migration to the HP environment, the relevant [TIGER/Line Shapefiles](#) were manually downloaded and preprocessed. Each shapefile was structured to include the desired metadata by decennial year as discrete layers. The pipeline supports both block-level and block group-level shapefiles: both contain polygon boundaries that associate block or block group, tract, county, and state codes with any longitude/latitude coordinate falling within a given polygon. Block group-level shapefiles are preferred for generating summary statistics due to their reduced complexity compared to block-level shapefiles.

In the 2026 Format workflow, two additional geographic codes representing Core Based Statistical Areas were associated using the same method: Metropolitan/Micropolitan Statistical Areas (denoted as CBSA) and Combined Statistical Areas (CSA). These designations were first introduced in 2003, replacing the Office of Management and Budget's (OMB) prior use of "Standard Metropolitan Areas." CBSA and CSA boundaries are typically updated annually, every five years at mid-decade, and following each decennial census. For the purposes of this processing, the 2010 and 2020 decennial releases were used alongside the earliest available annual release from the 2000s decade (the 2007 vintage) to provide coverage across all relevant time periods.

Additionally, ZIP Code Tabulation Areas (ZCTA) national files were obtained for each decennial year and compiled alongside the CBSA/CSA output. To speed up processing, the state acronym is joined during preparation of the compiled core areas file (containing the combined CBSA/CSA and ZCTA decennial details as layers).

Six types of census boundary shapefiles are used: state block-level, state block group-level, national CBSA, national CSA, national ZCTA, and national states (and equivalents). Each was obtained for the 2000, 2010, and 2020 decennial vintages, except CBSA and CSA, for which the 2007 vintage was substituted for 2000. These shapefiles were processed and compiled as GeoPackage (*.gpkg) files in "Data/Results/Census Bureau TIGER Line Shapefiles/", with state block- and block group-level data organized as layers by decennial year. CBSA, CSA, and ZCTA data (annotated with the state acronym) were compiled into a single file, with one layer per census boundary type and decennial year combination.

Unlike the CBSA and CSA shapefiles, the ZCTA shapefiles did not contain any state information. This was added via a spatial join with the state-level shapefile for the respective decennial year.

i Census Boundary Files: Download and Local Build Requirements

All census boundary files, both the downloaded raw shapefiles and the compiled GeoPackage versions, are too large for effective Git tracking and distribution. New users will need to download the source files independently and generate the GeoPackage files locally. For download instructions and links to sources, refer to the [Directory Specific Notes](#) in the project GitHub repository. For the coded implementation, refer to **SUBSECTION A3: Build Precompiled TIGER/Line GeoPackages** in "[Clean Raw Data_Step 2_2026 Format.R](#)".

Because this process is involved, a dedicated set of functions was created for the generation of these reference shapefiles: "[Support Functions/For Step 2_Compile Decennial Data.R](#)". The custom function `preflight_tiger_outputs()` first assesses the local environment for the status of shapefile downloads and builds, flagging any missing files or incomplete downloads to the user.

In brief, standardized decennial spatial layers for all census boundaries are compiled using `standardize_geo_layer()` for block and block group-level state GEOIDs, and `standardize_core_area_layer()` for CBSAs, CSAs, and ZCTAs. These standardized files are then written using `write_state_gpkg()` for GEOIDs and `write_core_areas_gpkg()` for CBSAs, CSAs, and ZCTAs, following the execution of

their respective `build_cbsa_csa()` and `build_zcta()` functions.

For size considerations, the GEOID files are retained separately by state, while the CBSA, CSA, and ZCTA layers are compiled as nationally representative files.

For each array run, the dataset is organized by state then ABI to mitigate the number of block group-level GeoPackages loaded simultaneously. GeoPackages for each state represented in that subset are imported using the custom function `read_state_gpkgs_for_data()` under **SUBSECTION B4: Load Relevant Block-Level GeoPackages by State**, creating one layer per state where, within each state, there is one layer per decennial period. Users must predefine the geography level — $\in c(\text{"blocks"}, \text{"block group"})$ — using `block_geography` to specify which level to utilize. The algorithm is configured to process either; while the block-level files contain block group information, processing only at the block group level yields faster performance.

The core areas shapefile contains national-level layers for all three areas (CBSA, CSA, and ZCTA) census boundaries across the three decennial periods. Prior to spatial joining, these files are filtered by the `area_states` column, which is extracted from `area_name` during the shapefile building process.

Spatial joining was conducted using the `sf` package via the custom functions `add_decennial_geoid_block()`, `decode_zcta()`, and `decode_cbsa_csa()`²⁴. Geocoordinates were converted to the standard World Geodetic System 1984 (WGS84) and projected into the coordinate reference system (CRS) of the census boundary polygons. A generalized outline of the algorithm executed by each function is outlined below.

Algorithm 2 High-Level Procedure: Spatial Join for Census/Area Identifier Assignment (CRS alignment; optional state partitioning)

Require: Points `cand_sf` (sf POINT) with unique `row_id` and optional state

Require: Polygon layer(s) containing boundary ID field(s) `id_col` (e.g., Census GEOID, block `geoid_block`, ZCTA/CBSA/CSA codes)

Ensure: Table keyed by `row_id` with attached ID(s) (often year-suffixed for decennial layers), NA if unmatched

1. **Hard-stop checks:** inputs are sf; `row_id` exists; `id_col` exists in polygons
2. **CRS match:** `pts` $\leftarrow$ `st_transform(cand_sf, st_crs(poly_sf))`
3. **Keep only needed polygon fields:** `poly_id` $\leftarrow$ `poly_sf[id_col]`
4. **Point-in-polygon join (left join):** `joined` $\leftarrow$ `st_join(pts, poly_id, join = st_within, left = TRUE)`
5. **Return:** drop geometry; output `row_id` and attached boundary IDs (e.g., `geoid_2000`, `geoid_2010`, `geoid_2020`)

Variations (mapped to functions):

add_decennial_geoid_block(): loop by **state** $\rightarrow$ loop by **year** (2000/2010/2020); join to each state-year layer; merge year-specific IDs by `row_id`.

decode_zcta(): use a **national** polygon layer; optionally pre-filter polygons by candidate states via an `area_states` field; then do a single join to return a ZCTA code column.

decode_cbsa_csa(): optionally pre-filter by `area_states`; split polygons into CBSA vs. CSA; perform **two** joins and return CBSA code/level plus CSA code (year-suffixed).

Addresses where no `address_line_1` was reported were reintroduced to the running candidate address dataset and, if valid geocoordinates were present, annotated with census boundaries. As noted above, this does not imply that the geocoordinates are valid for mapping closures.

Addresses were also annotated to indicate whether sufficient geocoordinates were available for GEOID joining and whether any match was found. Users should note that it is possible for addresses to not match with a core statistical area (CBSA or CSA), as these boundaries largely represent urban areas rather than rural ones.

LOOP PART E-I:

Up to this point, only a subset of the available columns had been evaluated in the preceding steps: address variables, summaries of year-open date ranges, geocoordinates, and all compiled validation results and quality control outcomes. The remainder of the loop rejoins this table with the rest of the metadata, which is then processed through three subsequent quality control assessments designed to generate insights at the time of execution.

These quality control tables also serve as the location where all raw values are presented alongside their relevant validation outcomes and quality control variables. Finally, all excess columns are consolidated into the final dataset form, prioritizing validated results over unvalidated versions and retaining a limited set of quality control columns to facilitate quick adjudication and back-tracking.

Refer to [Results - Step 2: Clean and Validate Addresses and Geolocation, and Annotate Records with Census Boundaries](#) for results on the three quality control assessment outcomes. For the implementation of these loop parts, refer to [“Code/Clean Raw Data_Step 2 HPC v2_2026 Format.R”](#). The input data consisted of the standardized and converted dataset generated in **Step 1**: `"./Data/Results/KEEP LOCAL/From Clean Raw Data/Step 1_2026 Format/church_2026_form_standardized_06.10.2026.parquet"`. Results were written to `"./Data/Results/KEEP LOCAL/From Clean Raw Data/Step 2_2026 Format/"`.

Compiling Results and Evaluating Import Quality

Results were saved in two columnar formats: Parquet and DuckDB^{3,25}. Parquet files are easy to handle during importation and exportation, but can only contain one dataframe at a time, much like a `*.csv` file. In contrast, DuckDB supports saving lists of dataframes, much like an `*.xlsx` file. To support the writing and reading of DuckDB files, custom functions were created for this purpose. Directories containing result Parquets, representing subset results from batch arrays, are compiled using `compile_parquet_folder()`. Directories containing result DuckDB lists are compiled using `compile_duckdb_folder()`.

Both functions also include generalized import quality checks (QC), which use the file name to verify that all expected ABIs are present in a defined subset. This check requires precise alignment between the dataset organization in the HPC script and the range defined in the file name, and allows for a quick confirmation that no information was lost across batch array runs. Additionally, `compile_parquet_folder()` generates summaries of the discrete QC outcomes (address validation and matching, census annotation, and geocoordinates) saved in the main

dataset alongside all results and metadata.

Two batches were run to complete the process described here. Any subsetting of the ABI search space, which defines which ABIs are processed during a given batch, is described in each respective section in **PART C: Recompile Results from the HPC**. Some QCs were also revised for conciseness and to account for variations not considered or covered by the HPC script. This was done in preparation for compiling the results across batches into the final result in **SUBSECTION C3: Combining the Batches**.

Primary datasets, comprising the main data with all metadata and quality control data, were compiled across batches, while import quality controls were maintained as separate lists for each batch. Batch-specific results are denoted by the first two digits of the batch number it is from. The main dataset was too large to compile in-memory, necessitating a DuckDB SQL compilation approach (code below). Results were then consolidated using the custom function `write_qc_groups()`, combining the compiled main data with their respective batch import quality control datasets.

RStudio Console

```
## SUBSECTION C3: Combining the Batches
# ~ line 1089-1141

# Open a writable DuckDB connection to the target database file.
con <- dbConnect(duckdb::duckdb(), dbdir = out_db, read_only = FALSE)

# Build/refresh "data" by unioning parquet batches (align by name) and
# ↪ sorting
# by ABI then archive_version_year (asc; NULLs last).
dbExecute(con, sprintf("
  CREATE OR REPLACE TABLE data AS
  SELECT *
  FROM (
    SELECT * FROM read_parquet('%s/**/*.parquet')
    UNION ALL BY NAME
    SELECT * FROM read_parquet('%s/**/*.parquet')
  )
  ORDER BY
  abi ASC,
  archive_version_year ASC NULLS LAST;
", p18, p20))
```

```

# Write QC outputs as separate tables named qc_df_18_* and qc_df_20_*
# (skips NULLs without templates).
write_qc_groups(
  con,
  list(
    qc_df_18 = qc_df_18$qc,
    qc_df_20 = qc_df_20$qc
  ),
  prefixes = c(qc_df_18 = "import_qc_18", qc_df_20 = "import_qc_20"),
  on_missing = "skip",
  verbose    = TRUE
)

# Closes the connection and persists changes to disk.
dbDisconnect(con, shutdown = TRUE)

```

The three at-point-of-analysis quality control datasets were smaller and easier to compile in memory. All outputs were saved using `write_list_to_duckdb()`, which also consolidated all datasets alongside their respective batch import quality control datasets.

For the implementation of data variable assessments relevant to the cleaning algorithm, post-evaluation importation, compiling, and checking, refer to [“Code/Clean Raw Data_Step 2_2026 Format.R”](#). The input data consisted of the standardized and converted dataset generated in **Step 1**: `"./Data/Results/KEEP LOCAL/From Clean Raw Data/Step 1_2026 Format/church_2026_form_standardized_06.10.2026.parquet"`. Results were written to `"./Data/Results/KEEP LOCAL/From Clean Raw Data/Step 2_2026 Format/"`.

Additional details regarding the algorithm’s performance outcomes can be found in [Results - Step 2: Clean and Validate Addresses and Geolocation, and Annotate Records with Census Boundaries](#), which includes results specifically assessing the outcomes of the three quality control assessments.

Step 3:

Results

Step 1: Nomenclature Standardization and Parquet Conversion

Processing the NAICS code columns was straightforward, involving the creation of new columns from the trailing two-digit proprietary codes and the renaming of the six-digit NAICS column names.

Testing the SIC code assumptions revealed points where standardization was needed. Overall, 18 description pairs with minor typographical variations were reconciled to a single version, 2 code redundancies were reduced, and 4 sources of NA descriptions were reconciled to existing descriptions for the same code.

Refer to Appendix - [Primary SIC Codes and Descriptions](#) All SIC Codes and Descriptions, and [Step 1 Final Result](#) for information on where to find the code that generates these results, as well as the results codebook.

Step 2: Clean and Validate Addresses and Geolocation, and Annotate Records with Census Boundaries

Compilation Quality Control Checks

Refer to Appendix - [Step 2 Final Result - Import Quality Checks](#), [State-Level Shapefile for Basemap](#), and [Missing Address Lines and PO Boxes By City](#) for information on where to find the code that generates these results, as well as the results codebook.

Results were compiled both from arrays within a batch and between batches. To ensure there was no data loss or erroneous introduction of duplications, basic importation QCs were generated. **SUBSECTION D2: Confirm Complete ABI Coverage** confirms that all imported data contains the needed ABIs — those that filed within the US. **SUBSECTION D4: At-Point-of-Evaluation Quality Checks** for the Address QC dataset further confirms that address validation and matching did not create any duplications of records.

Figure 3 and Figure 4 below summarize discrete outcomes for the different cleaning and validation steps conducted by array. Unlike in the respective QC datasets discussed in subsequent sections, these values are pulled from the main dataset where all ABIs are represented.

Figure 4 shows most entries resulted in a GEOID match (block group, tract, county, and state) averaging over 95%. On average, 80% of the geocoordinates used for spatial joining were verified, with less than 10% not comparable with the database due to missing `address_line_1`. Notably, the second batch array, where no address verification was allowed, yielded some arrays that had close to 100% unverified geocoordinates.

i Note

NAs for both verified and matched addresses were further resolved to those caused by addresses missing a valid `address_line_1` after importing. Results are not shown in these import QC tables.

Less than 20% of addresses were verified, likely attributable to the API query limitations imposed. Less than 5% of addresses were matched either exactly or fuzzy matched, with most matching occurring in the agnostic algorithm and negligible matching resulting from unverified addresses reconciled to verified ones.

Figure 3 resolves how many entries did not yield a match to any type of census boundary by decennial year. For some of these columns, such as the CBSA and CSA columns, missingness is expected since these census boundaries are assigned to urban locations. Both batches show very high annotation rates (low missingness) across the GEOID columns, with some limited missingness in the ZCTA columns. CBSA code and level missingness remains similar on average across the decennial years, but CSA code missingness attenuated from the 2000 to 2010 and again to the 2020 decennial period.

Overview of Address Completeness and ABI Filing Patterns

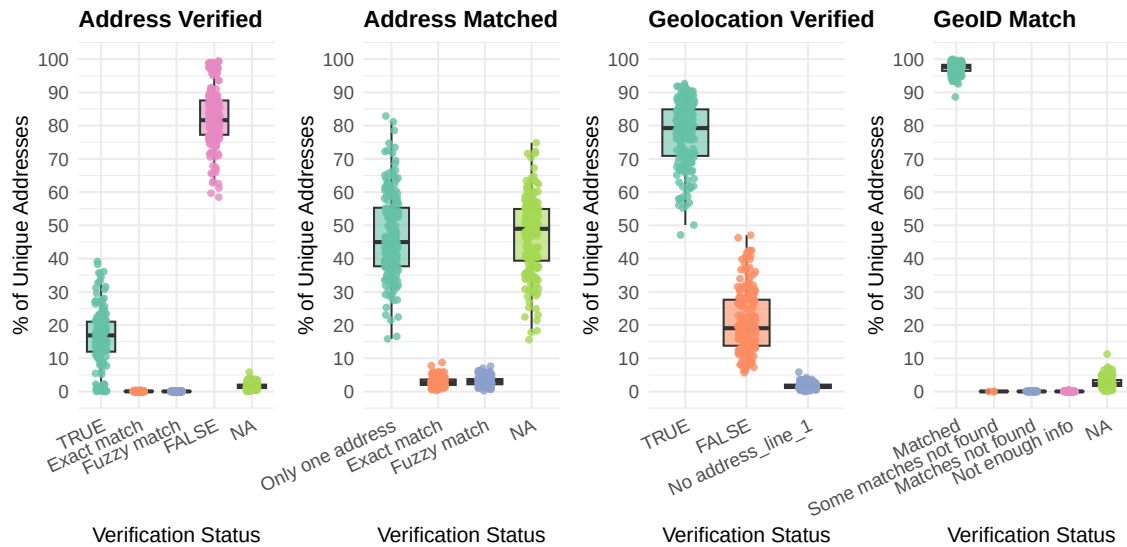
The cleaning and validation conducted in **Step 2** relied on a few core assumptions. Where possible, missing data or inconsistencies were handled by the algorithm, limiting the number of entries removed at this stage and maximizing the number of records rendered usable for downstream analysis. [Special Considerations for PO Boxes and Missing `address\_line\_1` Entries](#) examined the biggest factors to consider and outlined when critically missing values justify removing an entry from downstream analysis.

Table 3: Distribution of Number of Unique Addresses ABI Filed (counts reported in thousands)

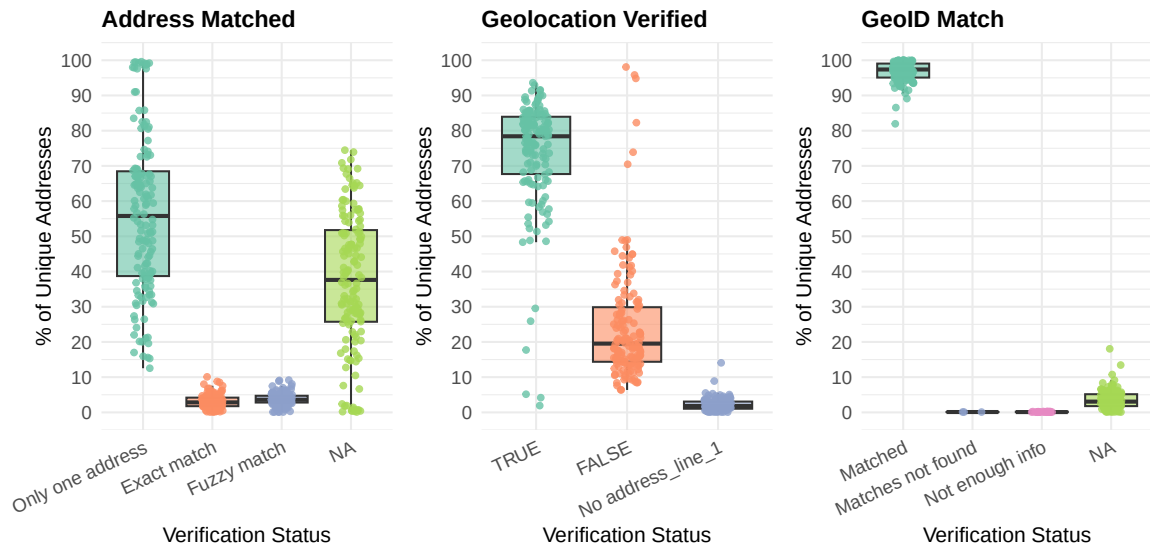
	1	2	3	4	5	6	7	8	9	10	11	12
Count (K)	759	205	74	28	10	3	0.900	0.300	0.050	0.010	0.002	0.002
Percent	70.21	19.00	6.88	2.59	0.91	0.29	0.09	0.02	0.00	0.00	0.00	0.00

QC Flag Distributions Across Arrays

Batch 18850425: 87% ABI coverage (allowed address verification)

**QC Flag Distributions Across Arrays**

Batch 20823868: Remaining ABI not covered in batch 18850425 (no address verification)

**Figure 3**

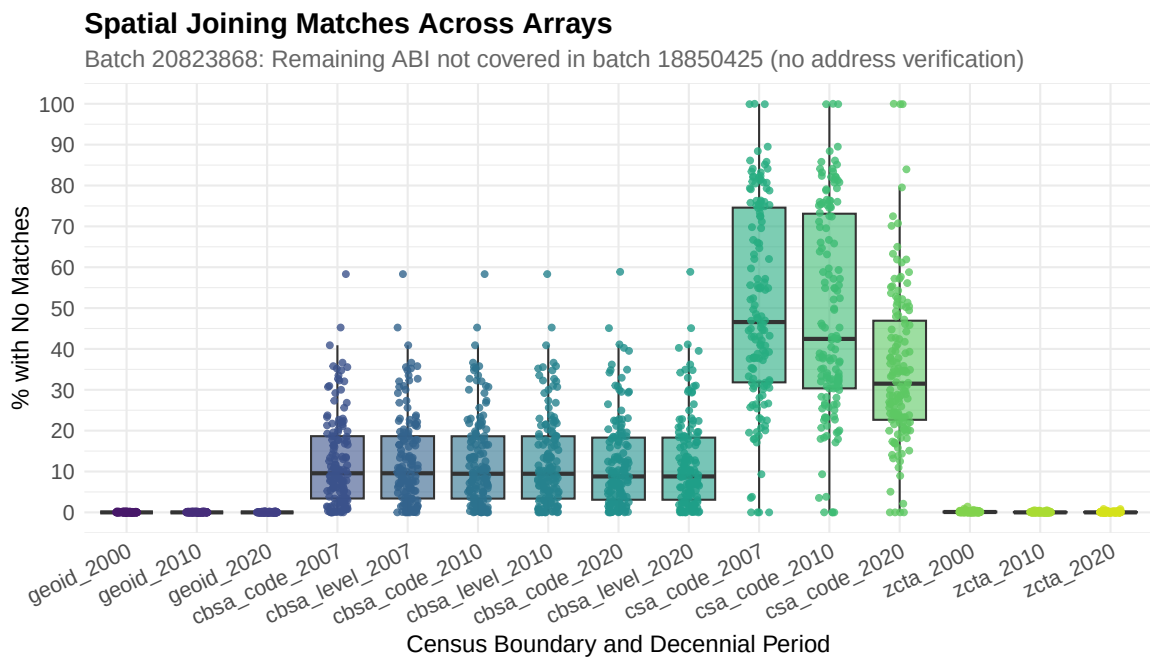
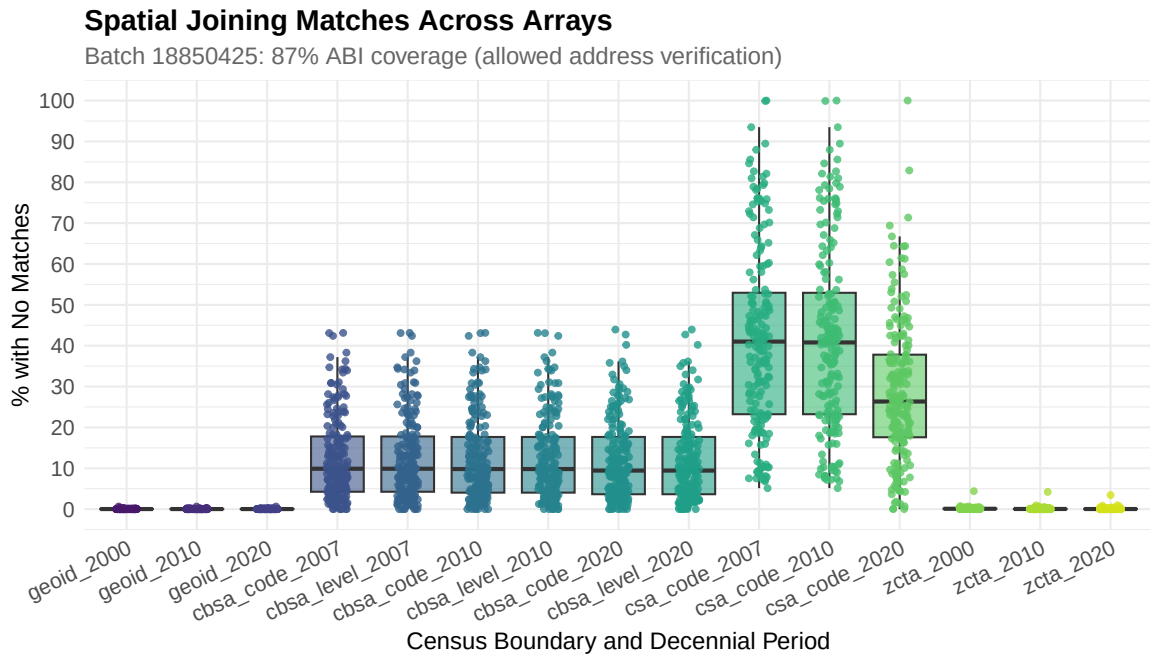


Figure 4

In addition, a few other variable characteristics were assessed and assumptions about them tested. All years reported are unique within a given ABI, making all ABI and address combinations unique for a given years-open value. 70% of ABIs filed under one unique address for the entire span of time, with 10% moving more than twice and less than 1% moving more than five times (Table 3).

City, state, five-digit ZIP code, and four-digit ZIP code extension are also given with an `address_line_1`. No entries are missing state or city; 0.001% lack a five-digit ZIP code and $\approx 22\%$ lack the four-digit ZIP extension. These missing values pose minimal geocoding risk, except for the 8 entries without a ZIP code, for which address validation failure would preclude SimpleMaps secondary validation. All entries have consistent formatting, assessed by string character length, for the state two-digit acronym, five-digit ZIP code, and four-digit extension, when reported.

Address Validation and Consolidation Process

Refer to Appendix - [Step 2 Algorithm Quality Check Datasets - Import Quality Checks](#) and [Step 2 Algorithm Quality Check Datasets - Addresses](#) for information on where to find the code that generates these results, as well as the results codebook.

The most comprehensive quality check (QC1) contains all ABIs; the remaining checks may show missing ABIs due to records being skipped during validation or matching (Table 4). Common reasons include NA values for `address_line_1` or no addresses remaining after USPS API validation. Additionally, approximately 70% of ABIs are expected to be ineligible for agnostic matching; without prior validation, these ABIs will not be captured by this metric. Proportions of ABIs that did not qualify for QC2 or QC3 were consistent across batches.

Table 4: Missingness test outcomes by QC key. **TRUE/FALSE** are counts of arrays passing/failing; **Min/Mean/Max** are the min/avg/max % of ABIs missing in those arrays.

import_qc_18						import_qc_20					
Key	FALSE	TRUE	Min	Mean	Max	Key	FALSE	TRUE	Min	Mean	Max
qc1	0	188	0.00	0.00	0.00	qc1	0	142	0.00	0.00	0.00
qc2	187	1	0.00	1.64	11.60	qc2	119	23	0.00	1.76	14.4
qc3	187	1	0.00	1.64	11.60						

QC1: API Usage, Matching Overview, and Unique Result Consistency

Of the records that underwent API validation, approximately 1.5% did not attempt verification — `verified_address` is absent from the dataset for these records (Table 5). All such cases correspond to a NA value in `address_line_1`; direct examination of these ABIs confirmed that no valid address was available, causing the validation step to be skipped.

Table 5: Verification Attempt by NA Address Presence and API Permission (%)

Allow Verification: FALSE			Allow Verification: TRUE		
Any NA Addresses	Attempted		Any NA Addresses	Attempted	
	FALSE	TRUE		FALSE	TRUE
FALSE	12.66	0.00	FALSE	0.00	84.71
TRUE	0.38	0.00	TRUE	1.43	0.82

Almost all ABIs (~98%) were processed through agnostic matching. No duplicates were detected following this step, and no address-year combinations differed between the raw and validated forms of the data, confirming that no data loss occurred.

QC2: Address Validation Using the USPS API

As described in [LOOP PART B: Consolidate and Verify the Addresses](#), unverified addresses were compared against verified ones using either exact or fuzzy matching. Exact matches on `address_line_1` bypassed the geolocation difference test entirely, while fuzzy matches required both coordinate differences to fall below 0.02 degrees (approximately 2 kilometers, or ≈ 1.4 miles). Of addresses that failed verification, fewer than 1% underwent fuzzy matching and subsequently failed the geolocation test (Table 6).

Table 6: Verified Address by Geolocation Test (% column)

Geolocation Test	Verified Address				
	TRUE	Exact Match	Fuzzy Match	FALSE	No <code>address_line_1</code>
TRUE	0.00	0.00	100.00	0.00	0.00
FALSE	0.00	0.00	0.00	0.11	0.00
Override	0.00	100.00	0.00	0.00	0.00
NA	100.00	0.00	0.00	99.89	100.00

Among addresses that were matched exactly or by fuzzy matching, most failed USPS

verification due to bad or access denied requests. Rate limiting accounted for fewer than one in five of these verification failures. In contrast, addresses that failed verification and could not be matched to any verified address were predominantly blocked by rate limits (Table 7).

Table 7: Verified Address by USPS Status Detail (% column)

USPS Status Detail	Verified Address				
	TRUE	Exact Match	Fuzzy Match	FALSE	No address_line_1
200 Successful operation	100.00	0.00	0.00	0.00	0.00
400 Bad request	0.00	77.56	82.87	1.54	0.00
403 Access denied	0.00	1.55	1.77	23.07	0.00
429 Rate limit reached	0.00	20.89	15.35	75.38	0.00
NA	0.00	0.00	0.00	0.00	100.00

The vast majority of successful verification attempts happened on the first try, with less than 1% verified using the secondary fallback strategy (Table 8). NOTE: All NAs correspond to outcomes where `attempt_succeeded == "No address_line_1"`.

Table 8: Attempt Succeeded by USPS Status Detail (% row)

USPS Status Detail	Attempt Succeeded		
	First Try	With City Correction by Zip5	None
200 Successful operation	99.92	0.08	0.00
400 Bad request	0.00	0.00	100.00
403 Access denied	0.00	0.00	100.00
429 Rate limit reached	0.00	0.00	100.00

QC3: Address Matching Results

Most fuzzy-matched attempts (86%) passed the geolocation test; fewer than 1% passed with an incomplete comparison due to one record missing geocoordinates (Table 9). This implies a small number of entries with missing geocoordinates were supplemented through string matching, in addition to any resolved through address-based geocoding.

Table 9: Geolocation Test Outcome by Matched Address (% row)

Matched Address	Geolocation Test			
	PASS	PASS; no Lon/Lat	FAIL	Override
Exact match	0.00	0.00	0.00	100.00
Fuzzy match	86.28	0.02	13.70	0.00
Only one address	0.00	0.00	0.00	0.00

Address-Based Geocoding

Refer to Appendix - [Step 2 Algorithm Quality Check Datasets - Import Quality Checks](#) and [Step 2 Algorithm Quality Check Datasets - Geocoordinates](#) for information on where to find the code that generates these results, as well as the results codebook.

The vast majority of arrays contained ABIs that were not processed through the geolocation tests (Table 10). This is expected, since QC1 and QC2 restricted their assessments to only those ABI records where an address was available for address-based geocoding. QC2 additionally filtered by non-missing verified geocoordinates. QC3 focused only on ABI records where more than one valid pair of geocoordinates was available for generating the summary tables.

Lack of representation is even higher for the second batch run, with maxima as high as 98% for QC2. However, the averages excluded still tend to be comparable with the previous batch. This is consistent with the results seen in earlier plots, where it was shown that this batch resulted in some arrays with high degrees of no validation (Figure 3 and Figure 4).

Table 10: Missingness test outcomes by QC key. **TRUE/FALSE** are counts of arrays passing/failing; **Min/Mean/Max** are the min/avg/max % of ABIs missing in those arrays.

import_qc_18						import_qc_20					
Key	FALSE	TRUE	Min	Mean	Max	Key	FALSE	TRUE	Min	Mean	Max
qc1	187	1	0.00	1.64	11.60	qc1	119	23	0.00	1.76	14.40
qc2	188	0	2.40	13.80	44.60	qc2	142	0	0.20	16.90	98.10
qc3	188	0	1.44	15.40	32.90	qc3	141	1	0.00	13.30	36.40

QC1: US Census Bureau Geocoder API Validation

A higher proportion of addresses with valid geocoordinates (non-NA) were verified through address-based geocoding. $\approx 80\%$ of entries without valid geocoordinates failed to verify in

comparison to $\approx 22\%$ of those with valid geocoordinates (Table 11).

Table 11: Enough Valid Geo by Geo Verified (% column)

Geo Verified	Enough Valid Geo	
	FALSE	TRUE
TRUE	10.30	78.12
FALSE	79.13	21.18
No address_line_1	10.57	0.70

“N Addresses” counts the number of unique addresses were aggregated and had valid geocoordinates (Table 12). In general, we see an increase in success as more records are included, but this drops down again with more than 7 records, with 8 having a 50/50 match rate.

Table 12: Geo Verified by Number of Addresses (% column)

Geo Verified	N Address								
	0	1	2	3	4	5	6	7	8
TRUE	10.30	76.04	85.52	87.58	87.00	88.21	85.86	70.83	50.00
FALSE	79.13	23.08	14.48	12.42	13.00	11.79	14.14	29.17	50.00
No address_line_1	10.57	0.88	0.00	0.00	0.00	0.00	0.00	0.00	0.00

As would be expected, any query where the number of attempts falls below the max number of benchmark/vintage tries listed successfully verified geocoordinates. $\approx 98\%$ of queries were successful with the first try. The next query tries with some successes was the third set, with only 0.02% verifying using the fourth set (Table 19). Recall the benchmark and vintage pairing tries listed under **SUBSECTION A3: Set Geocoder Search Priorities**:

Table 13: Benchmark and Vintage Try Pairs

benchmark_name	vintage_name
Public_AR_Census2020	Census2020_Census2020
Public_AR_Census2020	Census2010_Census2020
Public_AR_Current	Current_Current
Public_AR_ACS2025	Current_ACS2025

The first in queue, [Public_AR|Census2020]_Census2020, is expected to reference the same database as the third, [Public_AR|Current]_Current, supporting the higher matches

observed using both.

Table 14: Geolocation verification status by try attempt (% column)

# Attempts	Geo Verified		
	TRUE	FALSE	No address_line_1
1	97.70	0.00	0.00
2	0.10	0.00	0.00
3	2.18	0.00	0.00
4	0.02	100.00	0.00
<NA>	0.00	0.00	100.00

Of outcomes that reached four tries, 45 different combinations of API query responses were received, with 99.5% having all 200 status codes, indicating a successful query interaction. This confirms that failures to match largely were a result of no match being available in the respective benchmark/vintage try pairings databases used.

A string comparison was done between the queried address and the matched address in the US Census Bureau database. The algorithm processing JSON responses handled multiple possible matches through best match triaging. This assessment quickly identifies if the given address and best match chosen from the JSON response are exactly the same or similar ($\delta = 0.15$). Results show most JSON results were the exact same, 78% (Table 15). Almost 19% of responses that were not exactly the same were similar. Only 3% were neither similar nor the exact same.

Table 15: Query and JSON match address comparison (% column)

Address Similar	Address Same	
	FALSE	TRUE
FALSE	3.28	0.00
TRUE	18.70	78.02

It is important to assess if verifying addresses or agnostically matching them results in any changes with address-based geocoding. Results where a verification was retained (directly or through matching) show no obvious difference between those that had geocoordinates verified and those that did not (Table 16).

Table 16: Address verified and address matched outcomes by geolocation verified (% row)

Geocoordinates Verified	Address Verified			Address Matched		
	FALSE	Other	TRUE	FALSE	Other	TRUE
FALSE	75.10	15.60	9.33	60.90	31.20	7.90
No address_line_1	0.00	100.00	0.00	0.00	100.00	0.00
TRUE	74.00	12.80	13.10	41.60	46.20	12.20

For both verified addresses and agnostic matching, those compressed via fuzzy matching saw an attenuation in retrieving a match (Table 17). Exact matching did not seem to change results, neither compared with those directly verified by address nor with results having only one address (no matches attempted).

Table 17: Geo Verified by Address Verified/Matched (% row).

Address Verified	TRUE	FALSE	Address Matched	TRUE	FALSE
TRUE	83.86	16.14	Exact match	88.15	11.85
Exact match	86.91	13.09	Fuzzy match	80.78	19.22
Fuzzy match	65.43	34.57	Only one address	84.51	15.49
FALSE	78.41	21.59	<NA>	71.56	28.44
<NA>	74.63	24.57			

QC2: Reported vs. Validated Geocoordinate Comparison

It would be interesting to assess how well the reported geocoordinates aligned to the verified match from the Census Bureau Geocoder database. It was observed that 21% had a difference between at least one geocoordinate of more than 0.002 degrees (about 2 city blocks). 12% had a difference between both.

Table 18: Latitude/Longitude difference relative to 0.002 degrees (%)

Lat Diff > 0.002	Lon Diff > 0.002		
	FALSE	TRUE	<NA>
FALSE	78.72	5.49	0.00
TRUE	3.49	12.30	0.00
<NA>	0.00	0.00	0.00

When looking only at those where the differences exceeded 0.002 degrees, the mean difference is 0.0197 degrees and the max is 22.54 degrees in latitude; the mean difference is 0.0233 degrees and the max is 75.72 degrees in longitude. While most of the records were close to 2 kilometers or 1.4 miles difference, some were as many as 22 degrees, about the distance from New Haven, CT to Mexico City, Mexico, or even 75 degrees, about the distance from New Haven to London, England.

QC3: Geocoordinate Variation by Address

It would be interesting to assess how well the reported values aligned within the same reported address. Based on limited manual assessment, it was noted that these varied. Most addresses were within 0.02 degrees longitude or latitude, about 2 kilometers or 1.4 miles. About 5%, however, had some variation, either in longitude, latitude, or both (Table 19).

Table 19: Latitude/Longitude spread relative to 0.02 degrees (%).

Lat Spread > 0.02	Lon Spread > 0.02	
	FALSE	TRUE
FALSE	95.37	1.69
TRUE	1.02	1.92

One concern is that address validation or matching increased the amount of geolocation variation. However, based on the results from the previous quality check, this may be moot since the given coordinates have a large margin of error.

Looking at the latitude for verified addresses, there does not appear to be any strong effect on the max or average difference of geolocation amongst the same address. Similarly, for agnostically matched addresses, there does not appear to be a strong discernible influence on the max or average difference of geolocation amongst the same address (Table 20).

Looking again but for longitude, there also does not appear to be a strong influence caused by verifying the address or agnostically matching them (Table 20). For context, the number of results represented for each stratum is included. These run a little on the high end (Table 12).

Table 20: Lat/Lon spread summary (> 0.02) (subset where spread flag is TRUE)

Panel A: By Any Address Verified

Any Verified	Lat Spread > 0.02				Lon Spread > 0.02			
	n	Max Diff	Mean Diff	Avg N Geo	n	Max Diff	Mean Diff	Avg N Geo
FALSE	23433	51.8	0.0538	13	28926	110.0	0.0659	13
TRUE	3972	47.8	0.0678	14	4698	96.0	0.0835	13
<NA>	5660	47.4	0.0502	12	6997	102.0	0.0526	11

Panel B: By Any Address Matched

Any Matched	Lat Spread > 0.02				Lon Spread > 0.02			
	n	Max Diff	Mean Diff	Avg N Geo	n	Max Diff	Mean Diff	Avg N Geo
FALSE	839	46.0	0.0646	7	939	98.4	0.0813	7
TRUE	10741	45.7	0.0497	17	13256	99.6	0.0634	17
<NA>	21485	52.2	0.0570	11	26426	110.0	0.0662	11

Census Boundary Annotation Using Point-in-Polygon Spatial Joins

Refer to Appendix - [Step 2 Algorithm Quality Check Datasets - Import Quality Checks](#) and [Step 2 Algorithm Quality Check Datasets - Census](#) for information on where to find the code that generates these results, as well as the results codebook.

The vast majority of arrays contained ABIs that were not processed through the QC1 census tests assessing GEOIDs, where a higher proportion of arrays completely represented ABIs when assessing the CBSA/CSA/ZCTA boundaries (Table 21). This is expected, since QC1 restricted its assessments to only ABI records where enough geocoordinates were available for spatial joining. QC1 also filters by entries where a GEOID match was made after joining.

Lack of representation is slightly higher for the second batch run, with maxima as high as 19% for QC1 as opposed to 16%. However, the averages excluded still tend to be comparable with the previous batch. This is consistent with the results seen in earlier plots, where it was shown that this batch resulted in some arrays with high degrees of NA after spatial joining (Figure 3 and Figure 4).

Table 21: Missingness test outcomes by QC key. **TRUE/FALSE** are counts of arrays passing/failing; **Min/Mean/Max** are the min/avg/max % of ABIs missing in those arrays.

import_qc_18						import_qc_20					
Key	FALSE	TRUE	Min	Mean	Max	Key	FALSE	TRUE	Min	Mean	Max
qc1	188	0	0.02	2.61	16.30	qc1	126	16	0.00	2.91	19.0
qc2	77	111	0.00	0.03	0.64	qc2	26	116	0.00	0.03	0.30

QC1/2: GEOID and Core Statistics Vintage Alignment

A quick check comparing dimensions of distinct outcomes over address and over address with census boundaries showed each unique address is not consistently associated with the same census metadata for either QC1 or QC2. This result could be skewed due to the matching processes done. In general, we have reason to believe the same census boundaries were not applied to the same address over the years.

RStudio Console

```
## SUBSECTION D4: At-Point-of-Evaluation Quality Checks"

# GEOID columns ~ line 2909
n_distinct(census_qc$qc1$address) == nrow(distinct(census_qc$qc1, address,
  ↪ census_block))
n_distinct(census_qc$qc1$address) == nrow(distinct(census_qc$qc1, address,
  ↪ census_tract))
n_distinct(census_qc$qc1$address) == nrow(distinct(census_qc$qc1, address,
  ↪ county_code))
n_distinct(census_qc$qc1$address) == nrow(distinct(census_qc$qc1, address,
  ↪ fips_code))

# Statistical area columns ~ line 2944
n_distinct(census_qc$qc1$address) == nrow(distinct(census_qc$qc1, address,
  ↪ cbsa_level))
n_distinct(census_qc$qc1$address) == nrow(distinct(census_qc$qc1, address,
  ↪ cbsa_code))
n_distinct(census_qc$qc1$address) == nrow(distinct(census_qc$qc1, address,
  ↪ csa_code))
```

In this section, the big questions we want to answer are these: was the raw data annotated

with the correct census boundaries, and if not, what is the extent of the error; do the matched boundaries by decennial year correspond to the years the address was filed under. Note that the data provider did not include ZCTA columns, which is why these alignments are not checked here.

An attenuation of matching is observed as the boundary becomes smaller, with near perfect matching for FIPS and county. Only 0.01% of tract and block entries were reported as NA, and are consequentially uncheckable. CBSA code yielded a high match rate at 85%, CSA at 65%, and CBSA level at the lowest, with matches at 3%. Many CBSA level results were uncheckable too, at 12% (Table 22).

Table 22: Do raw values match any spatially joined decennial year equivalent? (% column)

census_qc\$qc1				census_qc\$qc2			
	Matched	None	Uncheckable		Matched	None	Uncheckable
FIPS	98.56	1.44	0.00	CBSA Code	85.87	14.13	0.00
County	97.63	2.37	0.00	CBSA Level	3.14	84.50	12.36
Tract	84.64	15.35	0.01	CSA	65.35	34.65	0.00
Block	84.16	15.82	0.01				

As would be expected, FIPS and county generally matched with GEOIDs across decennial periods (over 97%). Tract and block saw 44% and 35%, respectively. The next big category of matching was the 2000 decennial period, but nearly the same proportion of records match over the 2000, 2010 and 2010, 2020 decennial periods (Table 23). This indicates that the given GEOID boundaries do not clearly follow a pattern of annotating information from any one decennial period.

Most CBSA code, CBSA level, and CSA records did not match a decennial year, with a high level reporting as NA. The NA values arise from entries failing to annotate with that boundary during spatial joining, which is expected since these represent urban areas and will not cover the whole contiguous US. The vintage with the most matches was the 2020 vintage (as high as 14%), with all other vintages ranking well below 1% (Table 23).

Table 23: Records whose matched vintage set equals the given row label (% rows)

census_qc\$qc1					census_qc\$qc2			
Vintages	FIPS	County	Tract	Block	Vintages	CBSA Code	CBSA Level	CSA
2000	0.00	0.01	12.71	14.85	2000	0.00	0.00	0.00
2010	0.00	0.00	2.48	4.22	2010	0.00	0.00	0.02
2020	0.02	0.02	5.01	6.09	2020	5.40	0.47	13.53
2000, 2010	0.02	0.03	10.44	10.67	2000, 2010	0.00	0.00	0.00
2000, 2020	0.00	0.00	0.02	2.31	2000, 2020	0.00	0.00	0.00
2010, 2020	0.03	0.02	9.56	10.61	2010, 2020	0.33	0.01	0.01
2000, 2010, 2020	98.48	97.54	44.42	35.42	2000, 2010, 2020	0.00	0.00	0.00
None	1.44	2.37	15.35	15.82	None	14.13	84.50	34.65
Not reported	0.00	0.00	0.00	0.00	Not reported	0.00	0.00	0.00
Uncheckable	0.00	0.00	0.01	0.01	Uncheckable	0.00	12.36	0.00
					<NA>	80.13	2.66	51.79

The variability in census boundaries indicated earlier may legitimately be a manifestation of census boundaries getting correctly applied when that address was filed. However, we see that this assumption only holds for the most macro-level boundaries, FIPS and county. High adherence is also observed for CBSA codes, despite most matching with the 2020 decennial year (Table 24).

Granular boundaries like tract and block showed progressively increasing lack of alignment, with as high as 15% being unable to assess (Table 24). CSA had just more than half align, but users should note that this proportion may be misleading. We would expect that there is a high degree of failed comparisons since CSA is not applied for all parts of the US. Therefore, the real proportion of candidates is expected to be higher.

Table 24: Vintage-alignment outcomes with years open (% row)

census_qc\$qc1				census_qc\$qc2			
Boundary	FALSE	TRUE	<NA>	Key	FALSE	TRUE	<NA>
FIPS	0.04	98.52	1.44	CBSA Code	5.07	80.80	14.13
County	0.02	97.60	2.37	CBSA Level	1.22	1.92	96.86
Tract	17.55	67.09	15.36	CSA	12.11	53.24	34.65
Block	23.26	60.90	15.84				

Step 3:

Appendix

All SIC Codes and Descriptions

For the implementation, refer to **PART B: Correct the SIC Nomenclature** in “[Code/Clean Raw Data_Step 1_2026 Format.R](#)”. The data processed was the raw export `church_long_form_050926.csv`. Results were written to “`./Data/Results/KEEP LOCAL/From Process Data Update/Non-Primary SIC Codes_06.10.2026.csv`”.

Table 25: Codebook for Unique SIC Codes, Descriptions, and Column-Presence Flags (Post-Cleaning)

Field	Description
<code>sic_code</code>	The six-digit SIC code.
<code>sic_desc</code>	The description associated with the SIC code (following nomenclature cleaning).
<code>primary_sic_code...sic6_descriptions</code>	Boolean. TRUE if the code–description pair appears in the primary SIC code column; FALSE otherwise.
<code>sic_code...sic6_descriptions_sic</code>	Boolean. TRUE if the code–description pair appears in the first overflow SIC column; FALSE otherwise.
<code>sic_code_1...sic6_descriptions_sic1</code>	Boolean. TRUE if the code–description pair appears in the second overflow SIC column; FALSE otherwise.
<code>sic_code_2...sic6_descriptions_sic2</code>	Boolean. TRUE if the code–description pair appears in the third overflow SIC column; FALSE otherwise.
<code>sic_code_3...sic6_descriptions_sic3</code>	Boolean. TRUE if the code–description pair appears in the fourth overflow SIC column; FALSE otherwise.

Primary SIC Codes and Descriptions

For the implementation, refer to **PART B: Correct the SIC Nomenclature** in “[Code/Clean Raw Data_Step 1_2026 Format.R](#)”. The data processed was the raw export `church_long_form_050926.csv`. Results were written to `./Data/Results/KEEP LOCAL/From Process Data Update/Primary SIC Codes_06.10.2026.csv`.

Table 26: Codebook for Unique Primary SIC Codes and Descriptions (Post-Cleaning)

Field	Description
<code>primary_sic_code</code>	The six-digit primary SIC code.
<code>sic6_descriptions</code>	The description associated with the SIC code (after nomenclature cleaning).

Step 1 Final Result

For the implementation, refer to “[Code/Clean Raw Data_Step 1_2026 Format.R](#)”. The raw data from `./Data/Raw/KEEP LOCAL/church_long_form_050926.csv` were run through the nomenclature standardization and file type conversion outlined in **Step 1**. were run through the select nomenclature standardization and filetype conversion outlined in **Step 1**. Results were written to `./Data/Results/KEEP LOCAL/From Clean Raw Data/Step 1_2026 Format/church_2026_form_standardized_06.10.2026.parquet`.

Table 27: Codebook for new output fields produced by the evaluation.

Field	Description
<code>primary_naics6_code</code>	First six digits of the eight-digit NAICS code, representing the underlying NAICS classification (813110).
<code>naics6_descriptions</code>	Renamed form of the <code>naics8_descriptions</code> column.
<code>primary_naics2_code</code>	Last two digits of the eight-digit NAICS code. These are proprietary encodings added by Data Axle; no data dictionary was provided.

State-Level Shapefile for Basemap

For the implementation, refer to **SUBSECTION A2: Distribution of Characteristics Requiring Removal** in [“Code/Clean Raw Data_Step 1_2026 Format.R”](#). State-level census boundary files were incorporated using the `tigris` package ²⁶. Results were written to `"/Data/Results/From Clean Raw Data/Step 2_2026 Format/us_states_sf.shp"`.

Table 28: Codebook for `us_states_sf` 2023 Cartographic Boundaries Basemap

Field	Description
STATEFP	Two-digit state FIPS code (character).
STATENS	GNIS feature identifier for the state (character).
GEOIDFQ	Fully qualified geographic identifier (character).
GEOID	State geographic identifier (typically the state FIPS code as character).
STUSPS	USPS state abbreviation (e.g., “CA”) (character).
NAME	State name (character).
LSAD	Legal/statistical area description code (character).
ALAND	Land area in square meters (numeric/integer).
AWATER	Water area in square meters (numeric/integer).
geometry	State boundary geometry as MULTIPOLYGON.

Missing Address Lines and PO Boxes By City

For the implementation, refer to **SUBSECTION A2: Distribution of Characteristics Requiring Removal** in [“Code/Clean Raw Data_Step 2_2026 Format.R”](#). The **Step 1** standardized data were summarized by city (as recorded on the form or, when needed, validated via ZIP code using the SimpleMaps Basic [United States Cities Database](#) (version 1.90) lookup table created with the custom function `build_zip_city_lookup()` ⁹). These results were then joined to city geocoordinates from the `tigris` package ²⁶. Results showing missing `address_line_1` and the presence of PO Boxes were written separately to `"/Data/Results/From Clean Raw Data/Step 2_2026 Format/ABI with NA Addresses by City_08.07.2026.csv"` and `"/ABI with PO Boxes by City_08.07.2026.csv"`.

Table 29: Codebook for Evaluation Summary Output Fields (Missing `address_line_1` and PO Box Datasets)

Field	Description
<code>state/city/zip_code</code>	Address elements associated with each entry.
<code>n_abi</code>	Number of unique ABIs represented in that city or zip code.
<code>[n pct]_abi_any_na_address_line_1</code>	Count and percent of ABIs with at least one missing <code>address_line_1</code> .
<code>[n pct]_abi_poBox</code>	Count and percent of ABIs with at least one PO Box entry.
<code>lon/lat</code>	TIGER/Line Shapefile geocoordinates associated with the city.

GEOID Prebuild Shapefiles

For the implementation, refer to **SUBSECTION A4: Build Precompiled TIGER/Line GeoPackages** in “Code/Clean Raw Data_Step 2_2026 Format.R”. As described in the methods for **LOOP PART D: Point-in-Polygon Spatial Assignment of Census Information**, pulling in shapefiles using the `tigris` package was computationally prohibitive. Instead, relevant state-level shapefiles at the block and block-group level were manually downloaded from the Census Bureau TIGER/Line Shapefiles database (see [Directory Specific Notes](#) in the project GitHub repository for direct links to sources)²². These were then compiled into standardized shapefiles with layers for each decennial period (2000, 2010, and 2020) and written to the “./Data/Results/Census Bureau TIGER Line Shapefiles” directory.

Table 30: Codebook for State Block and Block-Group Shapefile Layers (2000/2010/2020)

Field	Description
<code>decennial_year</code>	The decennial census year associated with the boundary definition (2000, 2010, or 2020).
<code>geoid</code>	Full concatenated census geographic identifier, combining the state, county, tract, and block-group FIPS codes into a unique identifier for each boundary unit.
<code>statefp</code>	Two-digit state Federal Information Processing Series (FIPS) code identifying the state.
<code>countyfp</code>	Three-digit county FIPS code identifying the county within the state.
<code>tractce</code>	Six-digit census tract code identifying the census tract within the county.
<code>blkgrpce</code>	Single-digit block-group code identifying the block group within the census tract.
<code>geom</code>	Spatial geometry column containing the boundary polygons for each census unit, in the form of an <code>sf</code> geometry object.

Core Area CBSA/CSA Prebuild Shapefiles

For the implementation, refer to **SUBSECTION A4: Build Precompiled TIGER/Line GeoPackages** in “Code/Clean Raw Data_Step 2_2026 Format.R”. As described in the methods for **LOOP PART D: Point-in-Polygon Spatial Assignment of Census Information**, pulling in shapefiles using the `tigris` package was computationally prohibitive. Instead,

relevant state-level shapefiles for the Metropolitan and Micropolitan Statistical Areas (CBSA) and Combined Statistical Areas (CSA) were manually downloaded from the Census Bureau TIGER/Line Shapefiles database (see [Directory Specific Notes](#) in the project GitHub repository for direct links to sources)²². These were then compiled into standardized shapefiles with layers for each decennial period (2000, 2010, and 2020) and written to the `"./Data/Results/Census Bureau TIGER Line Shapefiles"` directory.

Table 31: Codebook for Core Area Shapefiles (CBSA/CSA; 2007/2010/2020 Layers)

Field	Description
<code>vintage_year</code>	The decennial census year associated with the boundary definition (2007, 2010, or 2020).
<code>area_type</code>	Classification of the area as either a Core-Based Statistical Area (CBSA) or a Combined Statistical Area (CSA).
<code>area_code</code>	Official Census Bureau numeric code uniquely identifying the CBSA or CSA.
<code>area_level</code>	Hierarchical classification of the CBSA, indicating whether the area is Metropolitan or Micropolitan; CSA entries are NA.
<code>area_name</code>	Full official name of the statistical area as designated by the Census Bureau (e.g., Atlanta-Sandy Springs-Alpharetta, GA).
<code>area_states</code>	Concatenated string of state abbreviations for all states encompassed by the statistical area boundary.
<code>geom</code>	Spatial geometry column containing the boundary polygons for each census unit, in the form of an <code>sf</code> geometry object.

Core Area ZCTA Prebuild Shapefiles

For the implementation, refer to **SUBSECTION A4: Build Precompiled TIGER/Line GeoPackages** in `"Code/Clean Raw Data_Step 2_2026 Format.R"`. As described in the methods for **LOOP PART D: Point-in-Polygon Spatial Assignment of Census Information**, pulling in shapefiles using the `tigris` package was computationally prohibitive. Instead, relevant state-level shapefiles for the ZIP Code Tabulation Areas (ZCTA) were manually downloaded from the Census Bureau TIGER/Line Shapefiles database (see [Directory Specific Notes](#) in the project GitHub repository for direct links to sources)²². These were then compiled into standardized shapefiles with layers for each decennial period (2000, 2010, and 2020) and written to the `"./Data/Results/Census Bureau TIGER Line Shapefiles"` directory.

Table 32: Codebook for ZCTA Shapefiles (2000/2010/2020 Layers)

Field	Description
vintage_year	The decennial census year associated with the boundary definition (2000, 2010, or 2020).
area_type	Classification of the area as ZCTA.
area_code	Official Census Bureau numeric code uniquely identifying the ZCTA.
area_states	Concatenated string of state abbreviations derived from a spatial join with the state-level TIGER/Line Shapefile for the respective decennial year.
geom	Spatial geometry column containing the boundary polygons for each census unit, in the form of an sf geometry object.

Step 2 Final Result

For the implementation, refer to “[Code/Clean Raw Data_Step 2_2026 Format.R](#)”. The standardized records generated in **Step 1** were run through the sequential cleaning and validation steps outlined in **Step 2**. Results were compiled across a batch’s array outputs and aggregated between batches into a single complete file in **PART C: Recompile Results from the HPC**, then written to “./Data/Results/KEEP LOCAL/From Clean Raw Data/Step 2_2026 Format/Compiled by Batches/church_2026_form_validated_08.03.2026.db”.

Table 33: Codebook for new output fields produced by the evaluation.

Field	Description
address	Best-available address: verified_address (trimmed; excluding “No address match found”), else matched_address, else combined_address.
address_verified	USPS-verified address result when available; NA if no match is found or verification did not succeed.
address_matched	Result from agnostic address matching when performed (e.g., exact or fuzzy match). “Only one address” indicates matching was attempted but only one address was reported.
reported_address	Original address as reported, prior to cleaning or verification.
longitude/latitude	Best-available coordinates: verified coordinates when available, otherwise averaged raw coordinates (stored as numeric).
geolocation_verified	Indicator that geocoordinates were verified (definition depends on the verification step used).
geoid_[2000 2010 2020]	Full census GEOID assigned via point-in-polygon spatial join with block-group TIGER/Line boundaries for the corresponding decennial year.
geoid_match	Summary of GEOID assignment across years: “Matched” (all years), “Some matches not found” (partial), “Matches not found” (none), or “Not enough info” (insufficient coordinate data).
cbsa_code_[2007 2010 2020]	CBSA numeric code assigned via point-in-polygon spatial join for the corresponding vintage year.
cbsa_level_[2007 2010 2020]	CBSA level (Metropolitan vs. Micropolitan) for the corresponding vintage year.
csa_code_[2007 2010 2020]	CSA numeric code assigned via point-in-polygon spatial join for the corresponding vintage year.
zcta_[2000 2010 2020]	Five-digit ZCTA assigned via point-in-polygon spatial join with ZCTA TIGER/Line boundaries for the corresponding decennial year.

Step 2 Final Result - Import Quality Checks

For the implementation, refer to “[Code/Clean Raw Data_Step 2_2026 Format.R](#)”. The standardized records generated in **Step 1** were run through the sequential cleaning and validation steps outlined in **Step 2**. Results were compiled across a batch’s array outputs and aggregated between batches into a single complete file in **PART C: Recompile Results from the HPC**.

Selected outcomes and characteristics of these within-batch imports were generated to enable rapid confirmation of expected coverage and completeness. These were written with the main results to “./Data/Results/KEEP LOCAL/From Clean Raw Data/Step 2_2026 Format/Compiled by Batches/church_2026_form_validated_08.03.2026.db”. Each batch’s results are annotated with the first two digits of the batch number (<XX>).

Table 34: Codebook for import_qc_<XX>\$abi_check

Field	Description
array	Batch array number from which the results were generated.
file	Source file name for the batch results.
from/to	ABI search-space index range defined under “SUBSECTION B1: Index Queue” in Clean Raw Data_Step 2 HPC v2_2026 Format.R.
n_expected_unique	Number of unique ABIs expected from the Step 1 data for the given index range.
n_actual_unique	Number of unique ABIs present in the batch results for the given index range.
n_missing	Number of ABIs absent from the batch results, computed as $ \text{setdiff}(\text{expected}, \text{actual}) $.
qc_pass	Boolean: TRUE if $n_{\text{missing}} = 0$, otherwise FALSE.

Table 35: Codebook for `import_qc_<XX>${address_matched|address_verified|geoid_match|geolocation_verified}`

Field	Description
array	Batch array number from which the results were generated.
value	Unique outcome observed for the QC column (e.g., "Exact match").
n	Number of records with the given outcome.
n_addr	Total number of addresses in the array.
pct	Percentage of addresses with the given outcome.

Table 36: Codebook for `import_qc_<XX>${na_census_boundaries}`

Field	Description
array	Batch array number from which the results were generated.
column	Variable checked for NA values.
n_addr	Total number of addresses in the array.
n_na	Number of NA entries observed for the column.
pct_na	Percentage of addresses with an NA for that census boundary column.

Step 2 Algorithm Quality Check Datasets - Import Quality Checks

For the implementation, refer to “[Code/Clean Raw Data_Step 2_2026 Format.R](#)”. The three quality control datasets — `address_qc`, `census_qc`, `geo_qc` — collected at the time of import were compiled from multiple HPC batches. A series of quality checks were run to confirm ABI coverage against the **Step 1** data. Each batch’s results are annotated with the first two digits of the batch number (<XX>). These were written with the main results to the `./Data/Results/KEEP LOCAL/From Clean Raw Data/Step 2_2026 Format/Compiled by Batches/` directory.

Table 37: Codebook for `import_qc_<XX>`

Field	Description
array	Batch array number from which the results were generated.
file	Source file name for the batch results.
key	Quality-check list (e.g., qc1, qc2) within the file from which the results were generated.
from/to	ABI search-space index range defined under “SUBSECTION B1: Index Queue” in Clean Raw Data_Step 2 HPC v2_2026 Format.R.
n_expected_unique	Number of unique ABIs expected from the Step 1 data for the given index range.
n_actual_unique	Number of unique ABIs present in the batch results for the given index range.
n_missing	Number of ABIs absent from the batch results, computed as $ \text{setdiff}(\text{expected}, \text{actual}) $.
qc_pass	Boolean: TRUE if $n_{\text{missing}} = 0$, otherwise FALSE.

Step 2 Algorithm Quality Check Datasets - Addresses

For the implementation, refer to “[Code/Clean Raw Data_Step 2_2026 Format.R](#)”. The three quality control (QC) datasets — address_qc, census_qc, geo_qc — collected at the time of import were compiled from multiple HPC batches. The address QC dataset included three checks covering the address validation and matching process:

- **QC1:** A general indicator of whether API verification was allowed, which ABIs were verified, whether any addresses were matched, and two checks for duplications or other introductions and losses of results.
- **QC2:** Summarizes all quality check outcomes from address validation using the USPS API over unique ABIs and addresses.
- **QC3:** Summarizes all quality check outcomes from address matching over unique ABIs and addresses.

These were written with the main results to the “./Data/Results/KEEP LOCAL/From Clean Raw Data/Step 2_2026 Format/Compiled by Batches/address_qc_08.05.2026.db

directory.

Table 38: Codebook for qc1

Field	Description
i	Index of the for loop iteration (results are stored as lists).
abi	Business ID to which the QC results pertain.
allow_usps_api	Boolean: TRUE if USPS API usage was permitted during that array run via <code>verify_addresses</code> ; otherwise FALSE.
api_used	Boolean: TRUE if the ABI entries are configured for USPS API usage via <code>do_api</code> ; otherwise FALSE.
verification_attempted	Boolean: TRUE if <code>verified_address</code> is present. This reiterates <code>do_api</code> and should match it.
match_attempted	Boolean: TRUE if agnostic matching was attempted for any addresses for that ABI, indicated by the presence of <code>matched_address</code> .
duplicates_induced	Boolean: TRUE if any duplicates over <code>combined_address</code> , <code>anchor_year_min</code> , and <code>anchor_year_max</code> are detected; otherwise FALSE.
any_addresses_line1_na	Boolean: TRUE if any <code>address_line_1</code> for that ABI is NA; otherwise FALSE.
new_not_in_old_addr_yr	From <code>compare_tabs()</code> : indicates whether the cleaned data contains any unique <code>combined_address</code> and <code>archive_version_year</code> outcomes not present in the Step 1 standardized data.
old_not_in_new_addr_yr	Inverse of <code>new_not_in_old_addr_yr</code> : indicates whether the Step 1 standardized data contains any outcomes not present in the cleaned data.
new_vs_old_differ	Boolean: TRUE if the processed data differs in any way from the Step 1 standardized form; otherwise FALSE.

Table 39: Codebook for qc2

Field	Description
i	Index of the for loop iteration (results are stored as lists).
abi	Business ID to which the QC results pertain.
address	Best-available address: <code>verified_address</code> (trimmed; excluding "No address match found"), else <code>matched_address</code> , else <code>combined_address</code> .
archive_versions_present	String listing all years in which the address was observed.
reported_address	Original reported address, prior to any cleaning or verification.
address_verified	If the address was verified against the USPS API, this field is TRUE. If unverified but a match is found, the value indicates the string-matching mode ("Override" or "Exact"). If no <code>address_line_1</code> was present for matching, this is indicated with "No <code>address_line_1</code> ".
ver_geolocation_test	Geolocation test for matching attempts to verified addresses: "Override" if "Exact" matched; TRUE/FALSE if "Fuzzy" matched; NA if no match was attempted.
usps_status	Numeric status code summarizing the USPS API query interaction (e.g., 200, 400).
usps_status_detail	Status description corresponding to <code>usps_status</code> (e.g., "200 Successful operation").
attempt_succeeded	Indicates whether verification succeeded on the first try, succeeded via fallback, or failed entirely.
verified_address	Verified address returned by the USPS API query.

Table 40: Codebook for qc3

Field	Description
i	Index of the for loop iteration (results are stored as lists).
abi	Business ID to which the QC results pertain.
address	Best-available address: <code>verified_address</code> (trimmed; excluding “No address match found”), else <code>matched_address</code> , else <code>combined_address</code> .
archive_versions_present	String listing all years in which the address was observed.
reported_address	Original reported address, prior to any cleaning or verification.
address_matched	If agnostic address matching was performed, this field records the outcome when matching succeeded via “Exact matching” or “Fuzzy matching”. “Only one address” indicates matching was attempted but only one address was reported.
match_geolocation_test	Geolocation test for agnostic matching attempts: “Override” if “Exact” matched; PASS/FAIL if “Fuzzy” matched. If PASS includes “no Lon”, “no Lat”, or “no Lon/Lat”, one record was missing that coordinate pair. NA if no match was attempted.
matched_address	Matched address result.

Step 2 Algorithm Quality Check Datasets - Census

For the implementation, refer to “[Code/Clean Raw Data_Step 2_2026 Format.R](#)”. The three quality control (QC) datasets — `address_qc`, `census_qc`, `geo_qc` — collected at the time of import were compiled from multiple HPC batches. **QC1** and **QC2** apply the same tests over different columns, verifying whether the reported census boundaries correspond with any of the spatial join results, and whether the matched vintages align with the dates the address was filed.

These were written with the main results to the “./Data/Results/KEEP LOCAL/From Clean Raw Data/Step 2_2026 Format/Compiled by Batches/census_qc_08.06.2026.db” directory.

Table 41: Codebook for qc1

Field	Description
i	Index of the for loop iteration (results are stored as lists).
abi	Business ID to which the QC results pertain.
address	Best-available address: <code>verified_address</code> (trimmed; excluding "No address match found"), else <code>matched_address</code> , else <code>combined_address</code> .
archive_versions_present	String listing all years in which the address was observed.
census_[block tract]/ [county fips]_code	Original reported census boundaries recorded for the address (block/tract and county/FIPS codes, as available).
n_address	Number of times the abi-address combination was recorded.
geoid_[2000 2010 2020]	Full concatenated census GEOID assigned via point-in-polygon spatial join with block-group TIGER/Line boundaries for the corresponding decennial year.
[fips county tract block] _code_any_match	Whether any original reported value matches any decennial value annotated via spatial joining: "Matched" if any do; "Uncheckable" if no original value was available; "None" if no matches.
[fips county tract block] _vintages	Which decennial vintages matched: "None" if none matched, or "Not reported" if no original value was reported.
[fips county tract block] _vintages_aligned	From <code>check_alignment()</code> : TRUE if matched vintages align with the years the address was reported; FALSE otherwise; NA if no comparison was available.
block_type	Level of the reported block boundary: "None reported" if missing; "Blocks" if four digits; "Block groups" if one digit; or "Not the expected dimensions" otherwise.

Table 42: Codebook for qc2

Field	Description
<code>i</code>	Index of the <code>for</code> loop iteration (results are stored as lists).
<code>abi</code>	Business ID to which the QC results pertain.
<code>address</code>	Best-available address: <code>verified_address</code> (trimmed, excluding "No address match found"), else <code>matched_address</code> , else <code>combined_address</code> .
<code>archive_versions_present</code>	String listing all years in which the address was observed.
<code>n_address</code>	Number of times the <code>abi</code> -address combination was recorded.
<code>cbsa_[level code]/csa_code</code>	Original reported CBSA/CSA boundaries for the address (CBSA level and/or CBSA code, and CSA code, as available).
<code>cbsa_code_[2007 2010 2020]</code>	Official Census Bureau CBSA numeric code assigned via point-in-polygon spatial join for the corresponding vintage year.
<code>cbsa_level_[2007 2010 2020]</code>	CBSA classification (Metropolitan vs. Micropolitan) for the corresponding vintage year.
<code>csa_code_[2007 2010 2020]</code>	Official Census Bureau CSA numeric code assigned via point-in-polygon spatial join for the corresponding vintage year.
<code>cbsa_[level code]_code_any_match</code> <code>/csa_code_code_any_match</code>	Whether any original reported value matches any vintage value annotated via spatial joining: "Matched" if any do; "Uncheckable" if no original value was available; "None" if no matches.
<code>cbsa_[level code]_vintages/</code> <code>csa_code_vintages</code>	Which vintages matched: "None" if none matched, or "Not reported" if no original value was reported.
<code>cbsa_[level code]_vintages</code> <code>_aligned/</code> <code>csa_code_vintages_aligned</code>	From <code>check_alignment()</code> : TRUE if matched vintages align with the years the address was reported; FALSE otherwise; NA if no comparison was available.

Step 2 Algorithm Quality Check Datasets - Geocoordinates

For the implementation, refer to “[Code/Clean Raw Data_Step 2_2026 Format.R](#)”. The three quality control (QC) datasets — `address_qc`, `census_qc`, `geo_qc` — collected at the time of import were compiled from multiple HPC batches. Three quality checks were conducted over the geocoordinate verification process:

- **QC1:** Summarizes all quality check outcomes from address-based geocoding validation using the US Census Bureau over unique ABIs and addresses.
- **QC2:** Assesses the difference between the reported and validated geocoordinates.
- **QC3:** Examines the spread of geocoordinates among all records sharing the same ABI and address, and indicates whether this summary includes records where the address was verified or agnostically matched.

These were written with the main results to the `"/Data/Results/KEEP LOCAL/From Clean Raw Data/Step 2_2026 Format/Compiled by Batches/geo_qc_08.06.2026.db` directory.

Table 43: Codebook for `qc1`

Field	Description
<code>i</code>	Index of the <code>for</code> loop iteration (results are stored as lists).
<code>abi</code>	Business ID to which the QC results pertain.
<code>address</code>	Best-available address: <code>verified_address</code> (trimmed; excluding “No address match found”), else <code>matched_address</code> , else <code>combined_address</code> .
<code>archive_versions_present</code>	String listing all years in which the address was observed.
<code>n_address</code>	Number of times the <code>abi</code> -address combination was recorded.
<code>enough_geo</code>	Boolean: TRUE if both longitude and latitude were reported; otherwise FALSE.
<code>geolocation_verified</code>	Boolean: TRUE if an address-based geocoding match was retrieved from the US Census Bureau Geocoder API; otherwise FALSE.

Field	Description
n_attempts	Number of API query attempts made; each attempt uses a different benchmark and vintage in the query URL.
query_statuses	Summary of returned query statuses. 200 indicates a successful interaction even if no match was found. Attempts are separated by a vertical bar.
all_200	Boolean: TRUE if all benchmark/vintage pairs returned status 200 (interaction succeeded but no match found); otherwise FALSE.
geo_matched_address	Matched address returned by the geocoding database.
matched_address_same	Uses <code>find_similar_addresses(threshold = 0)</code> to check whether the addresses are exactly the same.
matched_address_similar	Uses <code>find_similar_addresses(threshold = 0.15)</code> to check whether the addresses are similar.
benchmark/ vintage_input	Specific reference database (benchmark and vintage) used to search for an address match during geocoding.
address_verified	If the address was verified against the USPS API, this field is TRUE. If unverified but a match is found, the value indicates the string-matching mode ("Override" or "Exact"). If no <code>address_line_1</code> was present for matching, this is indicated with "No <code>address_line_1</code> ".
ver_geolocation_test	Geolocation test for matching attempts to verified addresses: "Override" if "Exact" matched; TRUE/FALSE if "Fuzzy" matched; NA if no match was attempted.
address_matched	If agnostic address matching was performed, this field records the outcome when matching succeeded via "Exact matching" or "Fuzzy matching". "Only one address" indicates matching was attempted but only one address was reported.
match_geolocation_test	Geolocation test for agnostic matching attempts: "Override" if "Exact" matched; PASS/FAIL if "Fuzzy" matched. If PASS includes "no Lon", "no Lat", or "no Lon/Lat", one record was missing that coordinate pair. NA if no match was attempted.

Table 44: Codebook for qc2

Field	Description
i	Index of the for loop iteration (results are stored as lists).
abi	Business ID to which the QC results pertain.
address	Best-available address: <code>verified_address</code> (trimmed; excluding “No address match found”), else <code>matched_address</code> , else <code>combined_address</code> .
archive_versions_present	String listing all years in which the address was observed.
[lat lon]_abs_diff	The absolute difference between the reported geocoordinate and the validated one.
[lat lon]_gt_002	Boolean: TRUE if the absolute difference exceeds 0.002 degrees; otherwise FALSE. NA if there were no geocoordinates to compare.

Table 45: Codebook for qc3

Field	Description
i	Index of the for loop iteration (results are stored as lists).
abi	Business ID to which the QC results pertain.
address	Best-available address: <code>verified_address</code> (trimmed; excluding “No address match found”), else <code>matched_address</code> , else <code>combined_address</code> .
n_address	Number of times the abi-address combination was recorded.
[lat lon]_ [min q1 median mean q3 max]	Summary statistics describing the spread of geocoordinates reported for a unique address.
[lat lon]_spread_gt_02	Boolean: TRUE if the spread of geocoordinates exceeds 0.02 degrees; otherwise FALSE. NA if there were no geocoordinates to compare.

Field	Description
<code>any_address_verified</code>	Boolean: TRUE if any address in these results came from a verified address; NA if no validation was used.
<code>any_address_matched</code>	Boolean: TRUE if any address in these results came from an agnostically matched address; NA if no matching was applied.

References

1. Ushey, K., Wickham, H. & Posit. [Introduction to renv](#).
2. Universal Service Administrative Co. (USAC). [Geolocation methods](#).
3. Neal Richardson et al. Arrow r package. <https://arrow.apache.org/docs/r/>.
4. U.S. Census Bureau. Census geocoder. <https://geocoding.geo.census.gov/geocoder/>.
5. United States Postal Service (USPS). Addresses 3.0 API. Developer portals. <https://developers.usps.com/addressesv3>.
6. harris-n-jalil-usps et al. [Api-examples](#). United States Postal Service (2026).
7. Wickham, H. & Posit Software, PBC. [Httr: Tools for working with URLs and HTTP](#). (2026).
8. Jeroen Ooms, Duncan Temple Lang & Lloyd Hilaiel. [Jsonlite: A simple and robust JSON parser and generator for r](#). (2025).
9. SimpleMaps. United states cities database. <https://simplemaps.com/data/us-cities>.
10. Mark P.J. van der Loo. The stringdist package for approximate string matching. Rdocumentation. <https://www.rdocumentation.org/packages/stringdist/versions/0.9.15/topics/stringdist> (2014) doi:10.32614/RJ-2014-011.
11. [Edit distance](#). Wikipedia (2026).
12. Srinivas Kulkarni. Jaro winkler vs levenshtein distance. Medium. <https://srinivas-kulkarni.medium.com/jaro-winkler-vs-levenshtein-distance-2eab21832fd6> (2021).
13. [Jaro-winkler distance](#). Wikipedia (2025).
14. [Big o notation](#). Wikipedia (2026).
15. DSA time complexity. https://www.w3schools.com/dsa/dsa_timecomplexity_theory.php.
16. [Adjacency list](#). Wikipedia (2026).
17. U.S. Geological Survey (USGS) & U.S. Department of the Interior. How much distance does a degree, minute, and second cover on your maps? <https://www.usgs.gov/faqs/how-much-distance-does-a-degree-minute-and-second-cover-your-maps> (2023).

18. National Oceanic and Atmospheric Administration (NOAA), National Hurricane Center & Central Pacific Hurricane Center. Latitude/longitude distance calculator. <https://www.nhc.noaa.gov/gccalc.shtml>.
19. [City block](#). Wikipedia.
20. [List of united states cities by area](#). Wikipedia.
21. U.S. Census Bureau. [Geocoding services web application programming interface \(API\)](#). (2026).
22. U.S. Census Bureau. TIGER/line shapefiles. Census.gov. <https://www.census.gov/geographies/mapping-files/time-series/geo/tiger-line-file.html> (2025).
23. Forrest, M. Spatial joins: A comprehensive guide - matt forrest. <https://forrest.nyc/spatial-joins-a-comprehensive-guide/> (2025).
24. Edzer Pebesma et al. Simple features for r. <https://r-spatial.github.io/sf/> (2018) doi:10.32614/RJ-2018-009.
25. Hannes Mühleisen, Mark Raasveldt & Kirill Müller. DBI package for the DuckDB database management system v. 1.5.5.9013. <https://r.duckdb.org/>.
26. Walker, K. & Rudis, B. [Tigris: Load census TIGER/line shapefiles](#). (2025).